

SoC 를 위한 *Peripheral* 설계

Reference : MicroBlaze.v15 [IHIL]



김 성 우

세미콘(*Semicon*)

대표

공학박사

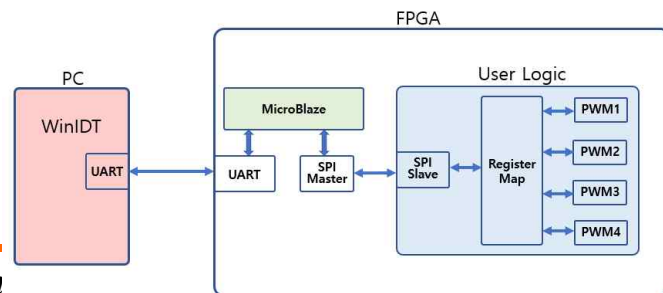
前 부경대 겸임교수

- ✓ 연락처 : 010 – 5509 – 3102
- ✓ 사무실 : 부산 남구 수영로 312, 1415호
- ✓ *ksemicon24@daum.net*

Table of Contents

➤ SoC를 위한 Peripheral 설계

1. Xilinx IP (Create Clock & Memory IP)
2. Create and Package New IP
3. SPI
 - 1) SPI Master
 - 2) SPI Slave
 - 3) SPI Controller
4. UART
5. AMBA
6. MicroBlaze_Hello World
7. MicroBlaze_LED_Counter
8. MicroBlaze_Peripheral Implementation
9. MicroBlaze_User Logic Interface

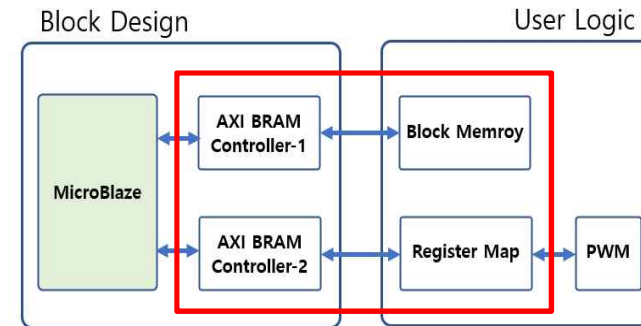


10. SPI_Master_IP(MicroBlaze_User Logic Interface)

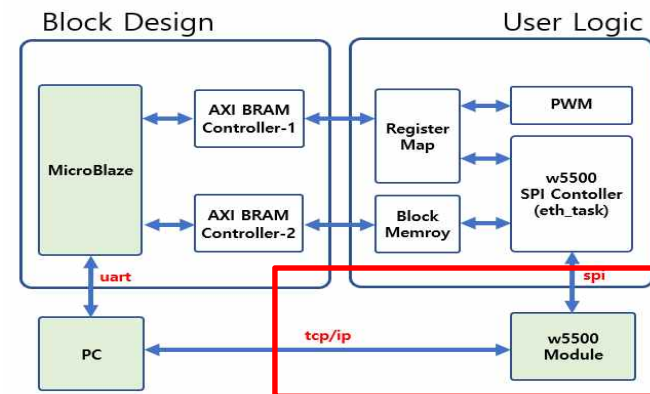
11. TCP_IP Implementation Using W5500

12. MicroBlaze_Block Memory Interface-1

13. MicroBlaze_Block Memory Interface-2



14. w5500 Interface Implementation(25_00_00_평가)



➤ SoC Peripheral RTC Design Project

Table of Contents

➤ SoC Peripheral RTC Design Project (25_00_00_프로젝트 계획서제출)

1. Project Exam

1) Exam_Washing_Machine

2) Exam_Microblaze_Washing_Machine_Block_Design

2. led_btn_Exam

3. AXI_My_IP_LED_Exam

4. AXI_My_IP_FND_Exam

5. AXI_RTC_Exam

6. AXI_PWM

7. AXI_LCD

8. Servo_Motor_Exam

9. DHT11_Module_Exam

10. Ultrasonic Module Exam

Table of Contents

➤ SoC를 위한 Peripheral 설계

1. Xilinx IP (Create Clock & Memory IP)

2. Create and Package New IP

3. SPI

- 1) SPI Master
- 2) SPI Slave
- 3) SPI Controller

4. UART

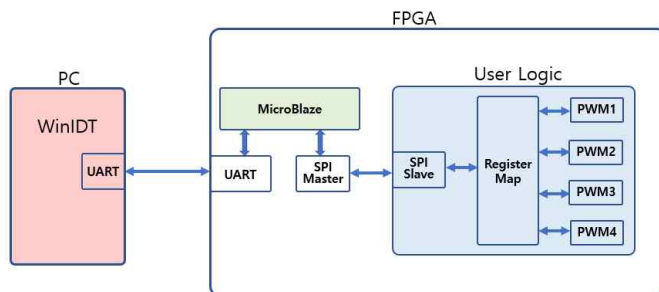
5. AMBA

6. MicroBlaze_Hello World

7. MicroBlaze_LED_Counter

8. MicroBlaze_Peripheral Implementation

9. MicroBlaze_User Logic Interface

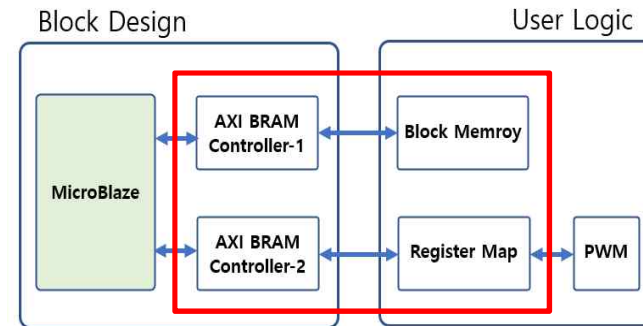


10. SPI_Master_IP(MicroBlaze_User Logic Interface)

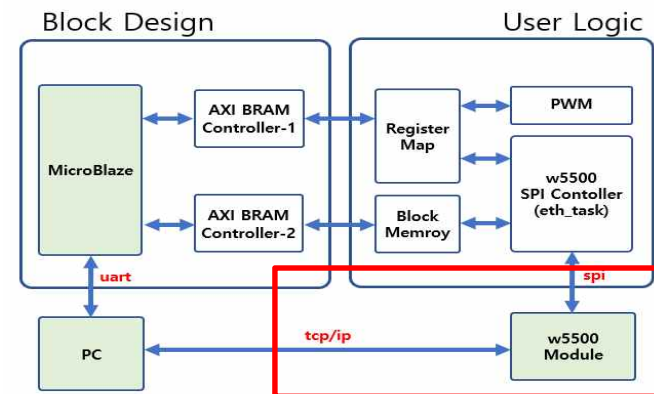
11. TCP_IP Implementation Using W5500

12. MicroBlaze_Block Memory Interface-1

13. MicroBlaze_Block Memory Interface-2



14. W5500 Interface Implementation(25_00_00_평가)



➤ SoC Peripheral RTC Design Project

Using Xilinx IP

➤ Create Clock

- *Clocking Wizard*를 통하여 자신이 원하는 *Clock* 을 생성
- *Clock Generation* → *Clock*을 이용하여 주파수(*Frequency*)와 듀티(*Duty*), 페이즈(*Phase*)를 원하는 값으로 변경 [$\text{Output Frequency} = \text{Input Frequency} * M / N(\text{분주비})$]

➤ Create Memory

- *Memory* 를 생성하는 *IP* : *FPGA* 내부의 Memory (*Block Memory*, *Distributed Memory*)를 사용하여 *Memory Block*을 생성하는 *IP*
- *Memory Interface* 를 생성하는 *IP* : *DDR* 과 같은 외부 *Memory* 와의 *Interface*를 생성하는 *IP*

Using Xilinx IP : Create Clock

Using Xilinx IP : Create Clock

➤ Create Clock

- Clocking Wizard를 통하여 자신이 원하는 Clock 을 생성
- Clock generation ➔ Clock을 이용하여 주파수(*Frequency*)와 듀티(*Duty*), 페이즈(*Phase*)를 원하는 값으로 변경 [$Output\ Frequency = Input\ Frequency * \{ M / N(분주비) \}$]

➤ Create Memory

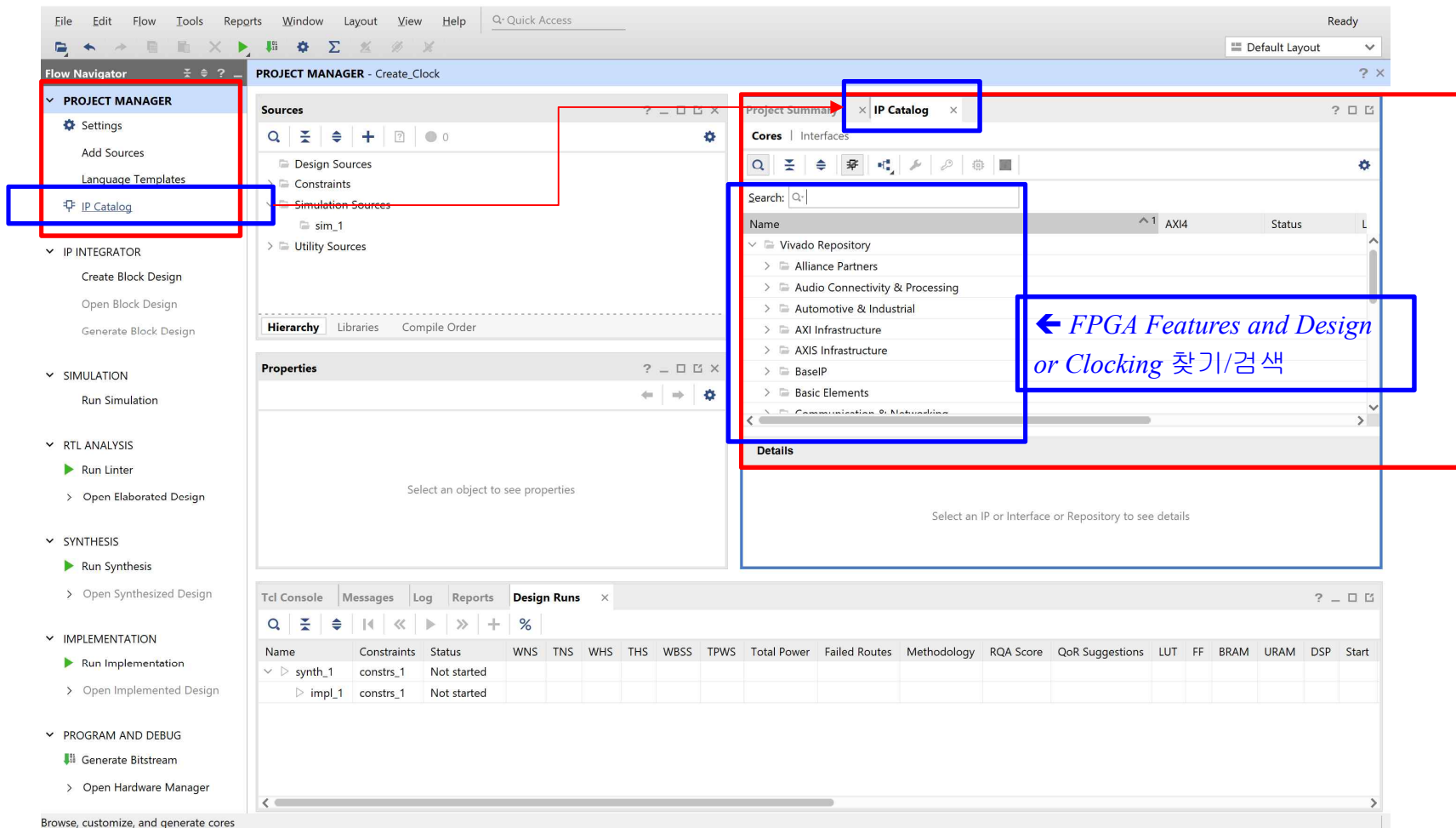
- Memory 를 생성하는 IP : FPGA 내부의 Memory (Block Memory, Distributed Memory)를 사용하여 Memory Block을 생성하는 IP
- Memory Interface 를 생성하는 IP : DDR 과 같은 외부 Memory 와의 Interface를 생성하는 IP

Using Xilinx IP : Create Clock

Create_Clock .xpr

➤ Create Clock

- ① [PROJECT MANAGER] → [IP Catalog] 클릭
- ② [FPGA Features and Design] → [Clocking] → [Clocking Wizard] : 더블 클릭



Using Xilinx IP : Create Clock

Create_Clock .xpr

➤ Create Clock

- ① [PROJECT MANAGER] → [IP Catalog] 클릭
- ② [FPGA Features and Design] → [Clocking] → [Clocking Wizard] : 더블 클릭 → Clock Wizard

The screenshot shows the Xilinx IP Catalog interface. The 'Cores' tab is selected, and the 'FPGA Features and Design' category is expanded. The 'Clocking Wizard' IP core is highlighted. The details panel on the left shows the following information:

- Name: Clocking Wizard
- Version: 6.0 (Rev. 12)
- Interfaces: AXI4
- Description: The Clocking Wizard creates an HDL file (Verilog or VHDL) that contains a clocking circuit customized to the user's clocking requirements.
- Status: Production
- License: Included
- Change Log: View Change Log
- Vendor: Xilinx, Inc.
- VLNv: xilinx.com:ip:clk_wiz:6.0
- Repository: E:/Xilinx/Vivado/2023.1/data/ip

The table of IP cores shows the following details for the Clocking Wizard:

Name	AXI4	Status	License	VLNV
Clocking Wizard	AXI4	Production	Included	xilinx.com:ip:clk_wiz:6.0

The Change Log for Clocking Wizard shows the following updates:

- 2023.1:
 - * Version 6.0 (Rev. 12)
 - * Bug Fix: CR Fixes
 - * Other: CR Fixes
- 2022.2.2:
 - * Version 6.0 (Rev. 11)
 - * No changes
- 2022.2.1:
 - * Version 6.0 (Rev. 11)
 - * No changes
- 2022.2:
 - * Version 6.0 (Rev. 11)
 - * Bug Fix: CR Fixes
 - * Other: CR Fixes
- 2022.1.2:
 - * Version 6.0 (Rev. 10)
 - * No changes

Annotations in the image provide additional context:

- AXI(Advanced eXtensible Interface)는 AMBA(Advanced Microcontroller Bus Architecture) 시스템의 한 종류로, IP 블록의 통신을 제어하게 하는 Protocol .
- Production IP Core : 일반 공개용으로 제공되며 검증된 코어
- IP 저장 위치

Using Xilinx IP : Create Clock

Create_Clock .xpr

➤ Create Clock

- ① [PROJECT MANAGER] → [IP Catalog] 클릭
- ② [FPGA Features and Design] → [Clocking] → [Clocking Wizard] : 더블 클릭
- ③ **Clock Wizard** → 더블 클릭 → [**Component Name**: clk_wiz_0]

Customize IP

Clocking Wizard (6.0)

Documentation IP Location Switch to Defaults

IP Symbol Resource

☐ Show disabled ports

reset clk_in1 clk_out1 locked

IP Symbol R ◀ ▶ ≡

☒ Show disabled ports

Component Name clk_wiz_0

Board **Clocking Options** **Output Clocks** **Port Renaming** **MMCM Settings** **Summary**

Associate IP interface with Clocking Options

IP Interface	Board Interface
CLK_IN1	Custom
CLK_IN2	Custom
EXT_RESET_IN	Custom

Clear Board Parameters

OK Cancel

Using Xilinx IP : Create Clock

Create_Clock .xpr

➤ Create Clock

- ① Clock Wizard ➔ 더블 클릭 ➔ [*Component Name : clk_wiz_0*]
- ② **Clocking Option** ➔ 모든 설정 : 기본값

Customize IP

Clocking Wizard (6.0)

Documentation IP Location Switch to Defaults

Component Name: clk_wiz_0

Board: **Clocking Options** Output Clocks Port Renaming MMCM Settings Summary

Clock Monitor

☐ Enable Clock Monitoring

Primitive

☒ MMCM ☐ PLL

Clocking Features

☒ Frequency Synthesis ☐ Minimize Power
☒ Phase Alignment ☐ Spread Spectrum
☐ Dynamic Reconfig ☐ Dynamic Phase Shift
☐ Safe Clock Startup

Jitter Optimization

☒ Balanced ☐ Minimize Output Jitter
☐ Maximize Input Jitter filtering

Dynamic Reconfig Interface

Opti~--- ☒ AXI4Lite ☐ DRP ☐ Phase Duty Cycle Config ☐ Write DRP registers

Input Clock Information

	Input Clock	Port Name	Input Frequency(MHz)	Jitter Options	Input Jitter	Source	
☒	Primary	clk_in1	100.000	10.000 - 800.000	UI	0.010	Single ended clock capable pin
☐	Secondary	clk_in2	100.000	60.000 - 120.000		0.010	Single ended clock capable pin

OK Cancel

Using Xilinx IP : Create Clock

Create_Clock .xpr

➤ Create Clock

① Clock Wizard → 더블 클릭 → [**Component Name : clk_wiz_0**]

② **Output Clocks** : [출력 Clock 3개 생성 (7개 까지 가능)] → [**Reset Type : Active Low**] → 그외 설정 : 기본값

❖ Clock Wizard 1개로 생성할 수 없는 Clock을 사용하는 방법 : Clock Wizard를 2단으로 사용

➔ Clock Wizard 는 **$Output_freq = Input_freq * M/N$** 의 식으로 출력 주파수를 생성

➔ Clock Wizard를 **2단으로 사용**하면, **$Output_freq = Input_freq * (M1/N1) * (M2/N2)$** 와 같이 구성
되어서 조금 더 복잡한 수식을 만들어 사용 가능

➔ Basys3 보드에는 **XC7A35T** 디바이스가 실장 : **XC7A35T** 는 최대 **5개까지 Clock Wizard**를 사용 가능

➔ **CMT(Clock Management Tiles)**의 개수가 사용 가능한 Clock Wizard의 수 Check

Artix-7 FPGA Feature Summary

Table 4: Artix-7 FPGA Feature Summary by Device

Device	Logic Cells	Configurable Logic Blocks (CLBs)		DSP48E1 Slices ⁽²⁾	Block RAM Blocks ⁽³⁾			CMTs ⁽⁴⁾	PCIe ⁽⁵⁾	GTPs	XADC Blocks	Total I/O Banks ⁽⁶⁾	Max User I/O ⁽⁷⁾
		Slices ⁽¹⁾	Max Distributed RAM (Kb)		18 Kb	36 Kb	Max (Kb)						
XC7A12T	12,800	2,000	171	40	40	20	720	3	1	2	1	3	150
XC7A15T	16,640	2,600	200	45	50	25	900	5	1	4	1	5	250
XC7A25T	23,360	3,650	313	80	90	45	1,620	3	1	4	1	3	150
XC7A35T	33,280	5,200	400	90	100	50	1,800	5	1	4	1	5	250
XC7A50T	52,160	8,150	600	120	150	75	2,700	5	1	4	1	5	250
XC7A75T	75,520	11,800	892	180	210	105	3,780	6	1	8	1	6	300
XC7A100T	101,440	15,850	1,188	240	270	135	4,860	6	1	8	1	6	300
XC7A200T	215,360	33,650	2,888	740	730	365	13,140	10	1	16	1	10	500

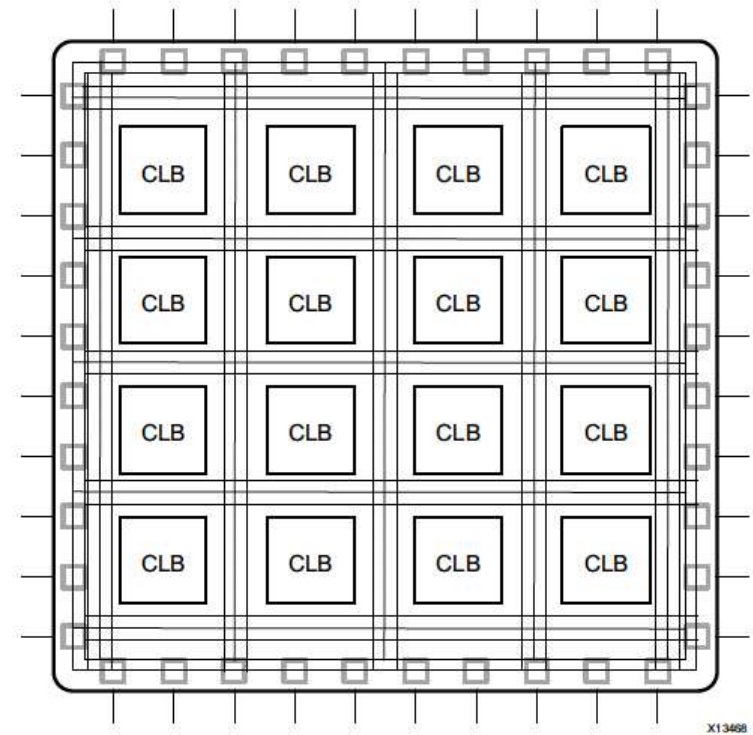
Notes:

- Each 7 series FPGA slice contains four LUTs and eight flip-flops; only some slices can use their LUTs as distributed RAM or SRLs.
- Each DSP slice contains a pre-adder, a 25 x 18 multiplier, an adder, and an accumulator.
- Block RAMs are fundamentally 36 Kb in size; each block can also be used as two independent 18 Kb blocks.
- Each CMT contains one MMCM and one PLL.
- Artix-7 FPGA interface blocks for PCI Express support up to x4 Gen 2.
- Does not include configuration Bank 0.
- This number does not include GTP transceivers.

Appendix

➤ Configurable Logic Blocks (CLBs)

- ① *Slice*
- ② *Flip-Flop*
- ③ *Look-up Table*
- ④ *Multiplexer*
- ⑤ *Carry Chain Block*



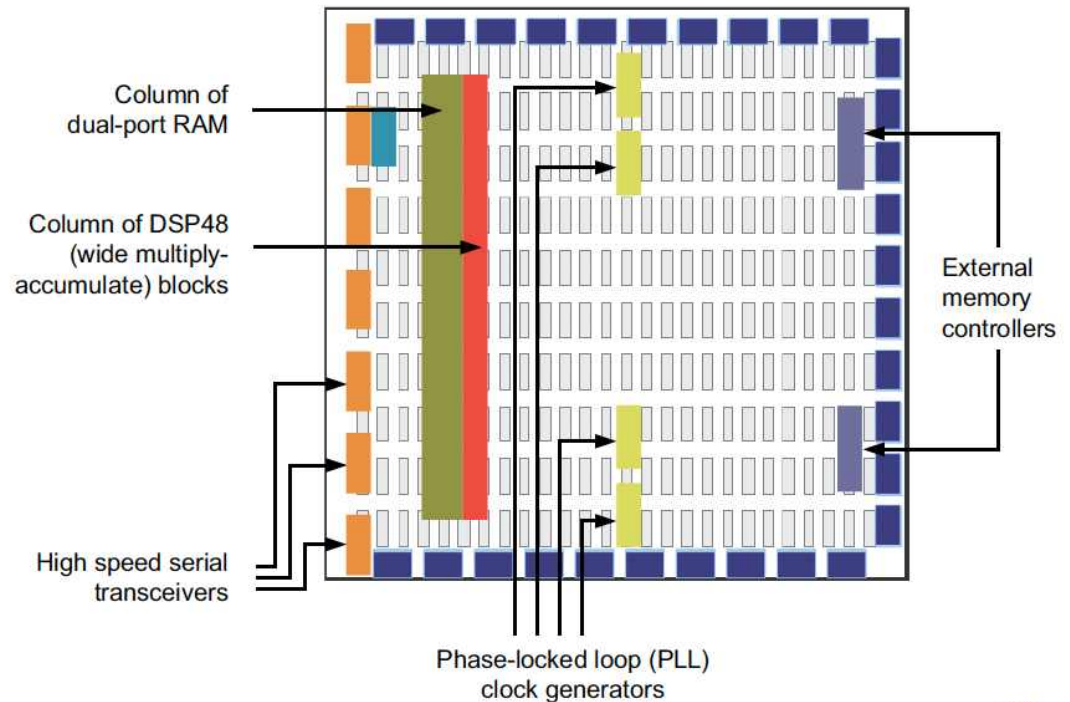
❖ Configurable Logic Blocks

- ➔ Configurable Logic Block(CLB)는 FPGA 설계의 핵심 구성 요소로, 설계자가 복잡한 디지털 회로를 원하는 대로 구현할 수 있도록 돕는 블록.
- ➔ CLB는 내부적으로 슬라이스(Slice)라는 작은 단위로 나뉘어 있으며, 이를 통해 설계자는 더 높은 유연성과 효율성을 확보. 그렇다면 왜 CLB는 이렇게 슬라이스로 나뉘어 있을까? ➔ 효율적인 자원 활용과 병렬 처리를 가능하게 하기 위한 설계.
- ➔ FPGA는 설계자가 원하는 논리 연산과 데이터 처리를 동시에 수행할 수 있는 병렬 구조를 제공. 그러나 CLB가 슬라이스로 나뉘어 있지 않고 하나의 거대한 블록으로만 구성되어 있다면, 간단한 연산을 수행하더라도 블록 전체가 점유되는 비효율적인 상황이 발생. 예를 들어, 작은 AND 연산 하나를 처리하려고 해도 CLB 전체가 사용된다면, 다른 연산을 수행할 자원이 부족해지고 FPGA 전체의 활용 효율이 크게 저하.
- ➔ 또한, 일부 논리 회로를 변경하거나 설계를 추가해야 하는 경우에도 큰 문제가 발생. CLB가 단일 블록으로만 구성되어 있다면, 작은 변경 사항이 있더라도 전체 블록을 수정하거나 재구성해야 하므로 설계 유연성이 떨어지고, 설계 시간이 불필요하게 늘어날 수 있음. 하지만 CLB를 여러 개의 슬라이스로 나누면 필요한 만큼만 자원을 사용할 수 있어, 병렬 처리와 부분적인 수정이 훨씬 용이

Appendix

➤ **Distributed RAM**

➔ 6-input LUT에 있는 64개의 LUT를 사용하면 64 비트를 저장할 수 있음. 한 슬라이스에는 8개의 6-input LUT가 있으므로 이를 사용하면 총 512비트의 값을 저장할 수도 있음.



X13468

➤ **BRAM (Block RAM)**

➔ Xilinx FPGA의 BRAM은 On-Chip 메모리로 FPGA의 fabric 상에 위치하는 dual-port RAM. 듀얼 포트 램이므로 한 사이클에 2개의 주소로부터 값을 읽거나 쓰기가 가능.

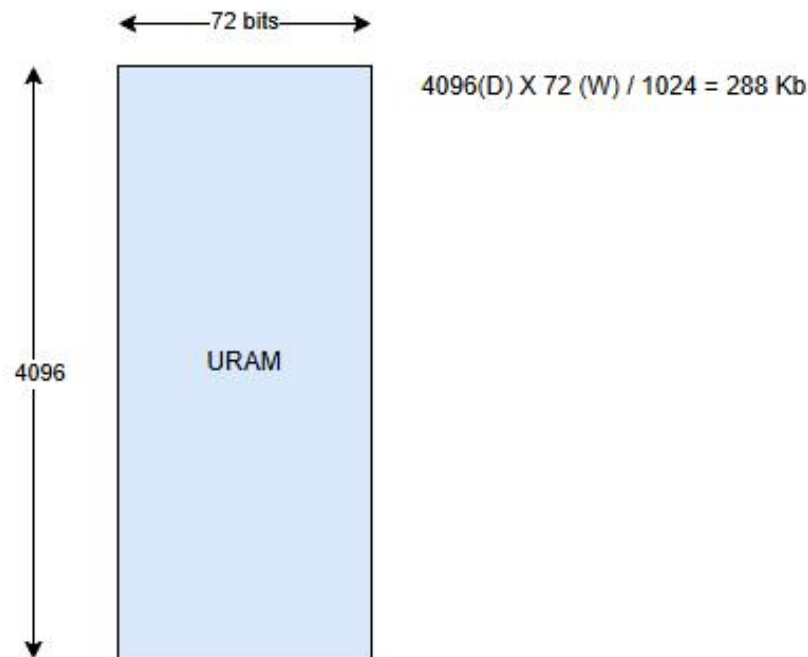
➔ FPGA 디바이스마다 다르지만 BRAM 하나는 18K 비트나 36K 비트를 저장할 수 있음. 여기서 36K는 36,000비트를 의미합니다. 이미지에서 한 픽셀이 8비트이므로 36,000비트이면 약 4,500 픽셀을 저장할 수 있는 양. 4,500 픽셀이면 매우 많아 보이지만 60X60 영역 정도에 해당. 스마트폰의 해상도가 FHD (1920X1080)임을 감안하면 60X60는 아주 작은 이미지.

➔ 참고로 BRAM과 URAM 모두 On-Chip 메모리. On-Chip 단어의 의미 그대로 메모리가 칩 밖에 있는게 아니라 칩 위에 있음을 의미다. 메모리가 칩 위에 있기 때문에 버스(Bus)를 사용하지 않고 바로 메모리에 접근할 수 있음.

Appendix

➤ *URAM (Ultra RAM)*

➔ Xilinx FPGA의 URAM은 UltraScale+ 이상의 제품군에만 있는 dual-port On-Chip 메모리. URAM은 288K 비트를 저장할 수 있는데 이는 BRAM 보다 8배 더 저장할 수 있는 양. URAM은 다음과 같이 메모리의 width와 depth가 고정되어 있습니다. URAM 한 개에는 8-bit 크기의 픽셀 36,000개를 저장할 수 있습니다.



Using Xilinx IP : Create Clock

Create_Clock .xpr

➤ Create Clock

- ① Clock Wizard ➔ 더블 클릭 ➔ [Component Name : clk_wiz_0]
- ② Output Clocks : [출력 Clock 3개 생성 (7개 까지 가능)] ➔ 테스트를 위해 100Mhz, 50Mhz, 200Mhz 생성
- ③ [Reset Type : Active Low] ➔ 그외 설정 : 기본값

The screenshot shows the Xilinx Clock Wizard IP configuration window for component 'clk_wiz_0'. The 'Output Clocks' tab is selected, showing a table of output clocks. Annotations include:

- A red box around the 'Output Clocks' tab header.
- A red box around the 'clk_out1' row in the 'Output Clocks' table.
- A red box around the 'BUFG' column header and the 'BUFG' entry for 'clk_out1'.
- A red box around the 'reset' port in the 'Port Renaming' section.
- A red box around the 'resetn' port in the 'Port Renaming' section.
- A red box around the 'Reset Type' section, showing 'Active Low' selected.
- A red box around the 'Enable Optional Inputs / Outputs for MMCM/PLL' section, showing 'reset' and 'locked' checked.

Output Clock	Port Name	Output Freq (MHz) Requested	Actual	Phase (degrees) Requested	Actual	Duty Cycle (%) Requested	Actual	Drives
☒ clk_out1	clk_out1	100.000	100.00000	0.000	0.000	50.000	50.0	BUFG
☐ clk_out2	clk_out2	100.000	N/A	0.000	N/A	50.000	N/A	BUFG
☐ clk_out3	clk_out3	100.000	N/A	0.000	N/A	50.000	N/A	BUFG
☐ clk_out4	clk_out4	100.000	N/A	0.000	N/A	50.000	N/A	BUFG
☐ clk_out5	clk_out5	100.000	N/A	0.000	N/A	50.000	N/A	BUFG
☐ clk_out6	clk_out6	100.000	N/A	0.000	N/A	50.000	N/A	BUFG
☐ clk_out7	clk_out7	100.000	N/A	0.000	N/A	50.000	N/A	BUFG

Output Clock	Sequence Number
clk_out1	1
clk_out2	1

Output Clock	Sequence Number
clk_out1	1
clk_out2	1
clk_out3	1
clk_out4	1
clk_out5	1
clk_out6	1
clk_out7	1

Reset Type: Active High ☐ Active Low ☒

Enable Optional Inputs / Outputs for MMCM/PLL: reset ☒ power_down ☐ input_clk_stopped ☐ locked ☒ clkfbstopped ☐

Using Xilinx IP : Create Clock

Create_Clock .xpr

➤ Create Clock

- ① Output Clocks : [출력 Clock 3개 생성 (7개 까지 가능)] ➔ 테스트를 위해 100Mhz, 50Mhz, 200Mhz 생성
- ② [Reset Type : Active Low] ➔ 그 외 설정 : 기본값
- ③ **MMCM Settings** : 출력 주파수를 설정하기 위하여 출력 = 입력 * M/N 으로 구성

✓ **Output_freq = Input_freq * (M/N)**

✓ **CLKFBOUT_MULT_F** 값이 **M**에 해당 & **Divide** 값이 **N**에 해당

✓ 1개의 Clock Wizard에서 출력되는 Clock들은 공통된 CLKFBOUT_MULT_F 값을 사용

✓ Divide 값은 각각의 출력 Clock에 적용

✓ $clk_out1 = 100M * 10 / 10 = 100 MHz$

✓ $clk_out2 = 100M * 10 / 20 = 50 MHz$

✓ $clk_out3 = 100M * 10 / 5 = 200 MHz$

Board | Clocking Options | Output Clocks | Port Renaming | **MMCM Settings** | Summary

These are the settings based on inputs from previous pages. Any update on this page will override the optimal settings calculated by the wizard

☐ Allow Override Mode

Attribute	Value
BANDWIDTH	OPTIMIZED
CLKFBOUT_MULT_F	10.000
CLKFBOUT_PHASE	0.000
CLKIN1_PERIOD	10.000
CLKIN2_PERIOD	10.000
COMPENSATION	ZHOLD
DIVCLK_DIVIDE	1
REF_JITTER1	0.010
REF_JITTER2	0.010
STARTUP_WAIT	☐
CLKFBOUT_USE_FINE_PS	☐
CLKOUT4_CASCADE	☐

Clk Wizard Port	Renamed Port	MMCM/PLL Port	Divide	Duty Cycle	Phase
clk_out1	clk_out1	CLKOUT0	10.000	0.500	0.000
clk_out2	clk_out2	CLKOUT1	20	0.500	0.000
clk_out3	clk_out3	CLKOUT2	5	0.500	0.000

Using Xilinx IP : Create Clock

Create_Clock .xpr

➤ Create Clock

- ① MMCM Settings : 출력 주파수를 설정하기 위하여 출력 = 입력 * M/N 으로 구성
- ② **Clock 의 Summary** : 입력 clock은 100 MHz, Single_ended 이고, 3개의 출력 clock이 생성

Component Name: clk_wiz_0

Board | Clocking Options | Output Clocks | Port Renaming | MMCM Settings | **Summary**

Primary Input Clock Attributes

Input Clock Frequency (MHz)	100.000
Clock Source	Single_ended_clock_capable_pin
Jitter	0.010

Clocking Primitive Attributes

Primitive Instantiated : MMCM
Divide Counter : 1
Mult Counter : 10.000
Clock Phase Shift : None

Clock Wiz O/p Pins	Source	Divider Value	Tspread (ps)	Pk-to-Pk Jitter (ps)	Phase Error (ps)
clk_out1	MMCM CLKOUT0	10.000	OFF	130.958	98.575
clk_out2	MMCM CLKOUT1	20	OFF	151.636	98.575
clk_out3	MMCM CLKOUT2	5	OFF	114.829	98.575
clk_out4	OFF	OFF	OFF	OFF	OFF
clk_out5	OFF	OFF	OFF	OFF	OFF
clk_out6	OFF	OFF	OFF	OFF	OFF
clk_out7	OFF	OFF	OFF	OFF	OFF

Using Xilinx IP : Create Clock

Create_Clock .xpr

➤ Create Clock

- ① Output Clocks : [출력 Clock 3개 생성 (7개 까지 가능)] ➔ [Reset Type : Active Low] ➔ 그외 설정 : 기본값
- ② [Port Renaming], [MMCM Settings], [Summary] ➔ 설정 : 기본값 ➔ OK
- ③ **Generate Output Products** ➔ Generate 클릭

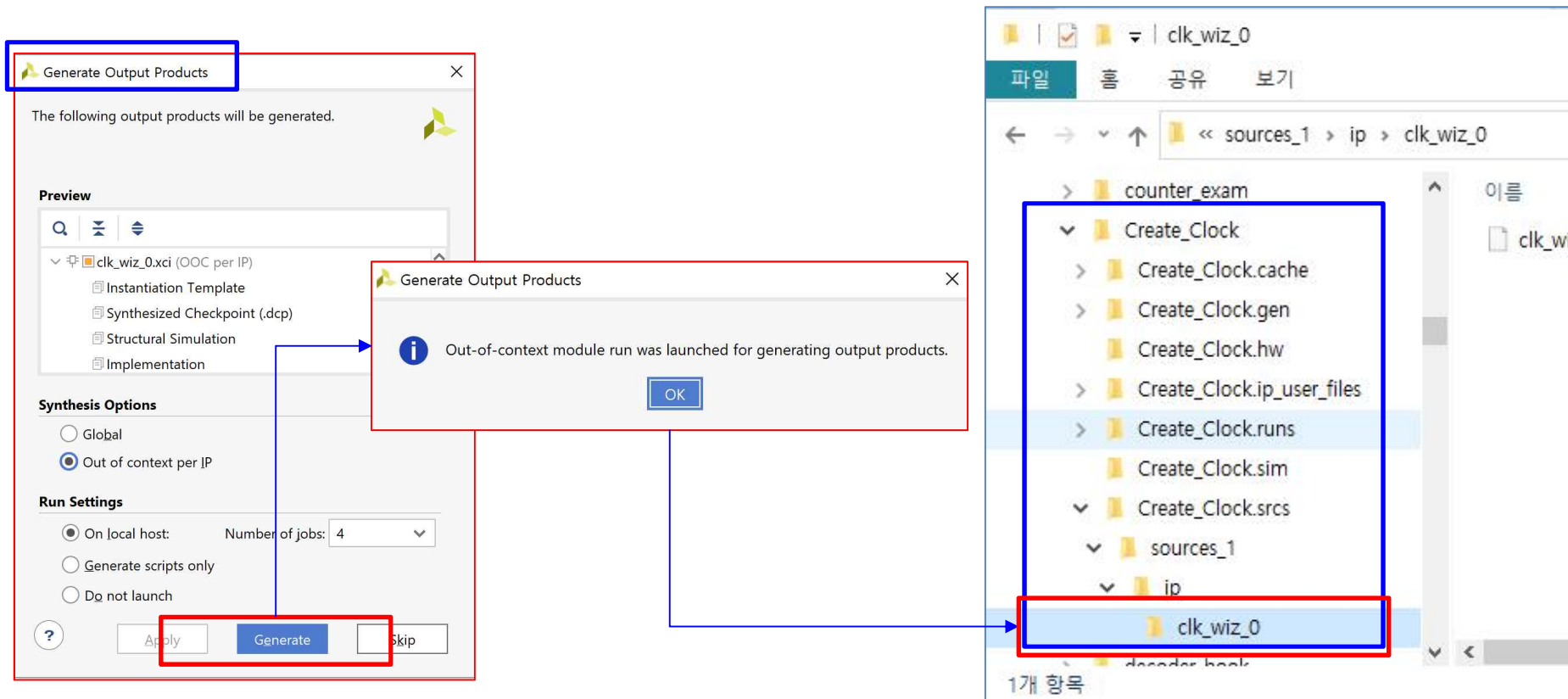
The image shows the Xilinx IP Configurator window for the 'clk_wiz_0' component. The 'Component Name' is 'clk_wiz_0'. The 'Clocking Options' tab is selected. The 'Generate Output Products' dialog box is open, showing the 'Preview' section with a list of output products: 'clk_wiz_0.xci (OOC per IP)', 'Instantiation Template', 'Synthesized Checkpoint (.dcp)', 'Structural Simulation', and 'Implementation'. The 'Synthesis Options' section has 'Out of context per IP' selected. The 'Run Settings' section has 'On local host' selected with 'Number of jobs' set to 4. The 'Generate' button is highlighted. A message box is also shown with the text 'Out-of-context module run was launched for generating output products.' and an 'OK' button.

Using Xilinx IP : Create Clock

Create_Clock .xpr

➤ Create Clock

- ① Generate Output Products → Generate 클릭
- ② IP 생성 → 저장 경로 [C:\Users\KIM\2023\Create_Clock\Create_Clock.srcs\sources_1\ip\clk_wiz_0]



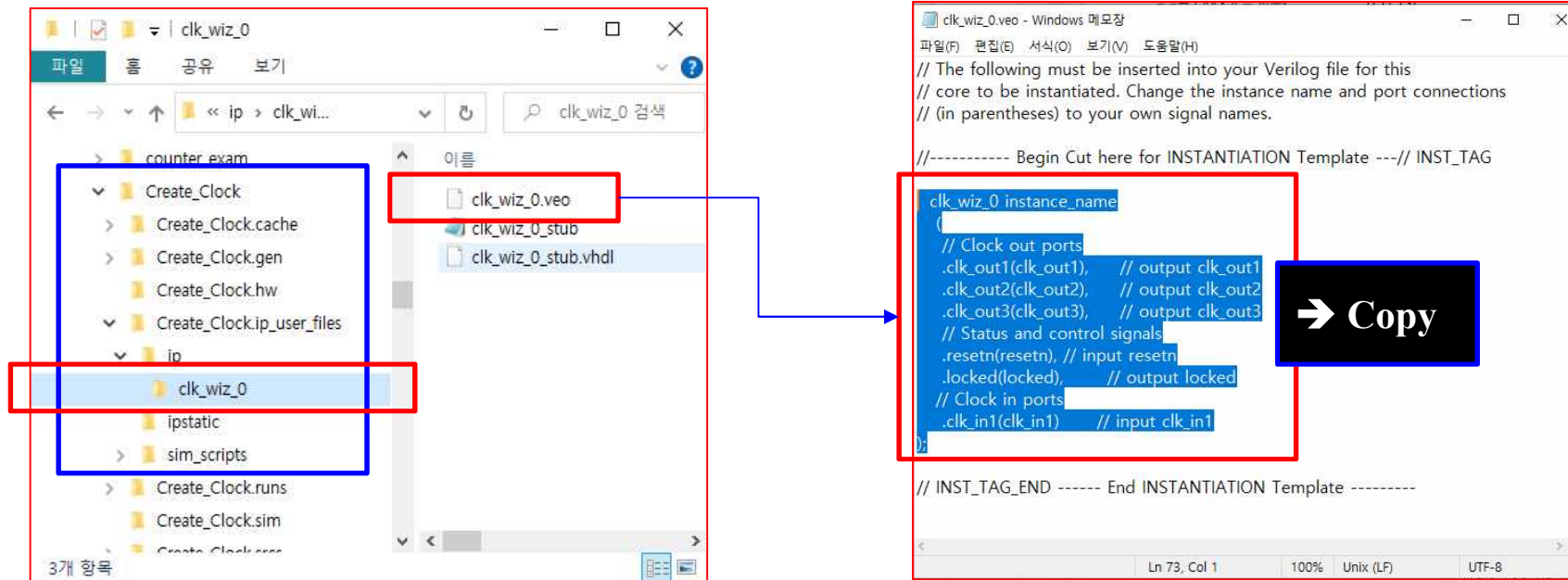
Using Xilinx IP : Create Clock

Create_Clock.xpr
Create_Clock_Exam.xpr

➤ Create Clock → Clock IP Test

✓ Create IP 사용

- `clk_wiz_0` : [C:\Users\KIM\2023\Create_Clock\Create_Clock.ip_user_files\ip\clk_wiz_0]
- “clk_gen.veo” file [← 생성한 IP를 사용하기 위한 내용] → Open → Copy



Using Xilinx IP

- ✓ *Create Clock IP*
- ✓ *Code Implementation for Clock IP Test (Case I)*
- ✓ *Code Implementation for Clock IP Test (Case II)*
- ✓ *Create Memory IP*

Using Xilinx IP : Create Clock

Create_Clock.xpr
Create_Clock_Exam.xpr

➤ Clock IP Test → Code Implementation

✓ Clock_counter.v → Design Source 생성 → led on/off 주기 변경 구현

- [1:0] sw 의 값에 따라 LED → On / Off 주기 변경
- [1:0] sw → 2'b00 → 200Mhz 로 동작 (LD0 : 0.25초 ~ LD3 : 2초)
- [1:0] sw → 2'b01 → 100Mhz 로 동작 (LD0 : 0.5초 ~ LD3 : 4초)
- [1:0] sw → 2'b10 → 50Mhz 로 동작 (LD0 : 1초 ~ LD3 : 8초)



Code Case I

clock_counter(1/3)

1. `timescale 1ns / 1ps

```
2. module clock_counter(resetn, clk, sw, led
3. );
4. input      resetn;
5. input      clk;
6. input [1:0] sw;
7. output [3:0] led;
```

→ 2비트 input sw

→ 4비트 output led

8. wire w_locked; → locked (high active), clock이 정상적으로 생성되었음을 알려주는 flag

9. wire w_clk200, w_clk100, w_clk50;

→ clk_gen에 의해서 생성되는 clock, 100Mhz, 50Mhz, 200Mhz

```
10. reg      [25:0] cnt0;
11. always @(posedge w_clk200 or negedge resetn)
12. begin
13. if(~resetn) cnt0 <= 26'd0;
14. else      cnt0 <= (cnt0==26'd50_000_000) ? 26'd0 : cnt0+1'b1;
15. end
```

→ sw → 2'b00 : 200Mhz 로 동작 (LD0 : 0.25초 ~ LD3 : 2초)

```
16. reg      [3:0] led0;
17. always @(posedge w_clk200 or negedge resetn)
18. begin
19. if(~resetn) led0 <= 4'b0;
20. else      led0 <= (cnt0==26'd50_000_000) ? led0 + 1'b1 : led0;
21. end
```

clock_counter(2/3)

```
22. reg      [25:0] cnt1;
23. always @(posedge w_clk100 or negedge resetn)
24. begin
25. if(~resetn) cnt1 <= 26'd0;
26. else      cnt1 <= (cnt1==26'd50_000_000) ? 26'd0 : cnt1+1'b1;
27. end
```

→ sw → 2'b01 : 100Mhz 로 동작 (LD0 : 0.5초 ~ LD3 : 4초)

```
28. reg      [3:0] led1;
29. always @(posedge w_clk100 or negedge resetn)
30. begin
31. if(~resetn) led1 <= 4'b0;
32. else      led1 <= (cnt1==26'd50_000_000) ? led1 + 1'b1 : led1;
33. end
```

```
34. reg      [25:0] cnt2;
35. always @(posedge w_clk50 or negedge resetn)
36. begin
37. if(~resetn) cnt2 <= 26'd0;
38. else      cnt2 <= (cnt2==26'd50_000_000) ? 26'd0 : cnt2+1'b1;
39. end
```

→ sw → 2'b10 : 50Mhz 로 동작 (LD0 : 1초 ~ LD3 : 8초)

```
40. reg      [3:0] led2;
41. always @(posedge w_clk50 or negedge resetn)
42. begin
43. if(~resetn) led2 <= 4'b0;
44. else      led2 <= (cnt2==26'd50_000_000) ? led2 + 1'b1 : led2;
45. end
```


Using Xilinx IP : Create Clock

Create_Clock .xpr
Create_Clock_Exam .xpr

➤ Clock IP Test ➔ Code Implementation

✓ Clock_counter.v ➔ Design Source 생성 ➔ led on/off 주기 변경 구현

- [1:0] sw 의 값에 따라 Led ➔ On / Off 주기 변경
- [1:0] sw ➔ 2'b00 ➔ 200Mhz 로 동작 (LD0 : 0.25초 ~ LD3 : 2초)
- [1:0] sw ➔ 2'b01 ➔ 100Mhz 로 동작 (LD0 : 0.5초 ~ LD3 : 4초)
- [1:0] sw ➔ 2'b10 ➔ 50Mhz 로 동작 (LD0 : 1초 ~ LD3 : 8초)

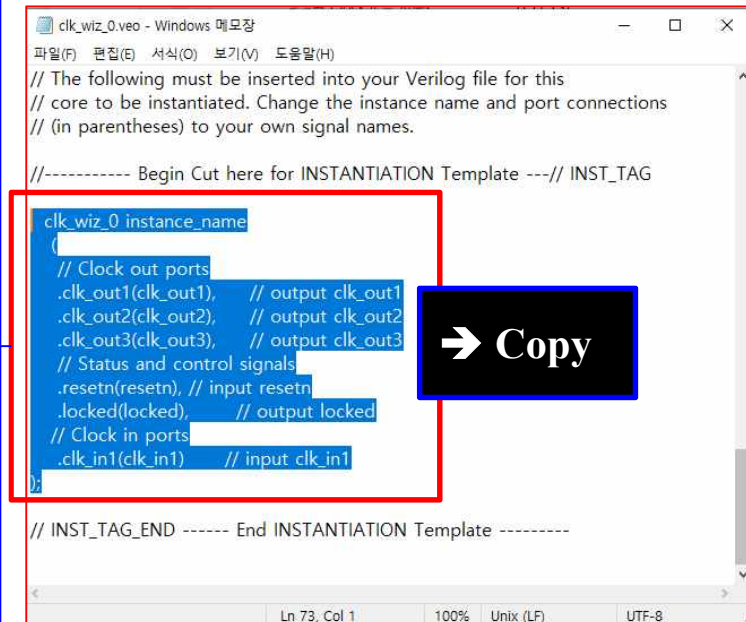
❖ Code Case I

clock_counter(3/3)

```
46. wire [3:0] led = (sw==2'b00) ? led0 : (sw==2'b01) ? led1 : led2;
```

```
47. clk_wiz_0 U0 ➔ clk_gen 모듈  
48. (  
49. // Clock out ports  
50. .clk_out1(w_clk100), // output clk_out1  
51. .clk_out2(w_clk50), // output clk_out2  
52. .clk_out3(w_clk200), // output clk_out3  
53. // Status and control signals  
54. .resetn(resetn), // input resetn  
55. .locked(w_locked), // output locked  
56. // Clock in ports  
57. .clk_in1(clk) //input clk_in1  
58.);
```

```
59. endmodule
```



Using Xilinx IP

- ✓ *Create Clock*
- ✓ *Code Implementation for Clock IP Test (Case I)*
- ✓ *Code Implementation for Clock IP Test (Case II)*
- ✓ *Create Memory IP*

Using Xilinx IP : Create Clock

Create_Clock .xpr
Create_Clock_Exam .xpr

➤ Clock IP Test → Code Implementation

✓ Clock_counter.v → Design Source 생성 → led on/off 주기 변경 구현

- [1:0] sw 의 값에 따라 Led → On / Off 주기 변경
- [1:0] sw → 2'b00 → 200Mhz 로 동작 (LD0 : 0.25초 ~ LD3 : 2초)
- [1:0] sw → 2'b01 → 100Mhz 로 동작 (LD0 : 0.5초 ~ LD3 : 4초)
- [1:0] sw → 2'b10 → 50Mhz 로 동작 (LD0 : 1초 ~ LD3 : 8초)



Code Case II

clockdivider

```
1. `timescale 1ns / 1ps

2. module clockdivider #(parameter N=50_000_000) (
3.     input clk,
4.     input reset,
5.     output reg clk1000Hz
6. );

7.     reg [25:0] cnt = 0;

8.     always @(posedge clk, negedge reset) begin
9.         if (~reset) begin
10.             cnt <= 0;

11.         end else begin
12.             if (cnt == (N - 1)) begin
13.                 cnt <= 0;
14.                 clk1000Hz <= ~clk1000Hz;
15.             end else begin
16.                 cnt <= cnt + 1;
17.             end
18.         end
19.     end

20. endmodule
```

counter

```
1. `timescale 1ns / 1ps

2. module counter (
3.     input inclk,
4.     input reset,
5.     output reg [3:0] out_counter
6. );

7.     always @(posedge inclk, negedge reset) begin
8.         if (~reset) begin
9.             out_counter <= 0;
10.         end else begin
11.             out_counter <= out_counter + 1;
12.         end

13.     end

14. endmodule
```

Using Xilinx IP : Create Clock

Create_Clock .xpr
Create_Clock_Exam .xpr

➤ Clock IP Test → Code Implementation

✓ Clock_counter.v → Design Source 생성 → led on/off 주기 변경 구현

- [1:0] sw 의 값에 따라 Led → On / Off 주기 변경
- [1:0] sw → 2'b00 → 200Mhz 로 동작 (LD0 : 0.25초 ~ LD3 : 2초)
- [1:0] sw → 2'b01 → 100Mhz 로 동작 (LD0 : 0.5초 ~ LD3 : 4초)
- [1:0] sw → 2'b10 → 50Mhz 로 동작 (LD0 : 1초 ~ LD3 : 8초)



Code Case II

clock_counter(1/2)

```
1. `timescale 1ns / 1ps
2. module clock_counter(resetn, clk, sw, led
3. );
4. input      resetn;
5. input      clk;
6. input [1:0] sw;
7. output [3:0] led;
8. wire [3:0] led0, led1, led2;
9. wire w_locked;
10. wire w_clk200, w_clk100, w_clk50;
11. wire w_clk2hz, w_clk1hz, w_clk1_2hz;
12. wire [3:0] led = (sw==2'b00) ? led0 : (sw==2'b01) ? led1 : led2;
13. clockdivider #(N(50_000_000)) U0 (
14. .clk(w_clk200),
15. .reset(resetn),
16. .clk1000Hz(w_clk2hz)
17. );
18. counter U1 (
19. .inclk(w_clk2hz),
20. .reset(resetn),
21. .out_counter(led0)
22. );
```

clock_counter(2/2)

```
1. clockdivider #(N(50_000_000)) U2 (
2. .clk(w_clk100),
3. .reset(resetn),
4. .clk1000Hz(w_clk1hz)
5. );
6. counter U3 (
7. .inclk(w_clk1hz),
8. .reset(resetn),
9. .out_counter(led1)
10. );
11. clockdivider #(N(50_000_000)) U4 (
12. .clk(w_clk100),
13. .reset(resetn),
14. .clk1000Hz(w_clk1_2hz)
15. );
16. counter U5 (
17. .inclk(w_clk1_2hz),
18. .reset(resetn),
19. .out_counter(led2)
20. );
```

Using Xilinx IP : Create Clock

Create_Clock.xpr
Create_Clock_Exam.xpr

➤ Clock IP Test ➔ Code Implementation

✓ Clock_counter.v ➔ Design Source 생성 ➔ led on/off 주기 변경 구현

- [1:0] sw 의 값에 따라 Led ➔ On / Off 주기 변경
- [1:0] sw ➔ 2'b00 ➔ 200Mhz 로 동작 (LD0 : 0.25초 ~ LD3 : 2초)
- [1:0] sw ➔ 2'b01 ➔ 100Mhz 로 동작 (LD0 : 0.5초 ~ LD3 : 4초)
- [1:0] sw ➔ 2'b10 ➔ 50Mhz 로 동작 (LD0 : 1초 ~ LD3 : 8초)



Code Case II

clock_counter(3)

```
1. clk_wiz_0 U6
2. (
3.   // Clock out ports
4.   .clk_out1(w_clk100), // output clk_out1
5.   .clk_out2(w_clk50),  // output clk_out2
6.   .clk_out3(w_clk200), // output clk_out3
7.   // Status and control signals
8.   .resetn(resetn), // input resetn
9.   .locked(w_locked), // output locked
10.  // Clock in ports
11.  .clk_in1(clk) // input clk_in1
12. );
13. endmodule
```

PROJECT MANAGER - Create_Clock_Exam

Sources

Design Sources (1)

- clock_counter (clock_counter.v) (7)
 - U0 : clockdivider (clockdivider.v)
 - U1 : counter (counter.v)
 - U2 : clockdivider (clockdivider.v)
 - U3 : counter (counter.v)
 - U4 : clockdivider (clockdivider.v)
 - U5 : counter (counter.v)
- U6 : clk_wiz_0 (clk_wiz_0.xci)

```
clk_wiz_0 instance_name
(
  // Clock out ports
  .clk_out1(clk_out1), // output clk_out1
  .clk_out2(clk_out2), // output clk_out2
  .clk_out3(clk_out3), // output clk_out3
  // Status and control signals
  .resetn(resetn), // input resetn
  .locked(locked), // output locked
  // Clock in ports
  .clk_in1(clk_in1) // input clk_in1
);

// INST_TAG_END ----- End INSTANTIATION Template -----
```

➔ Copy

Using Xilinx IP : Create Clock

Create_Clock .xpr
Create_Clock_Exam .xpr

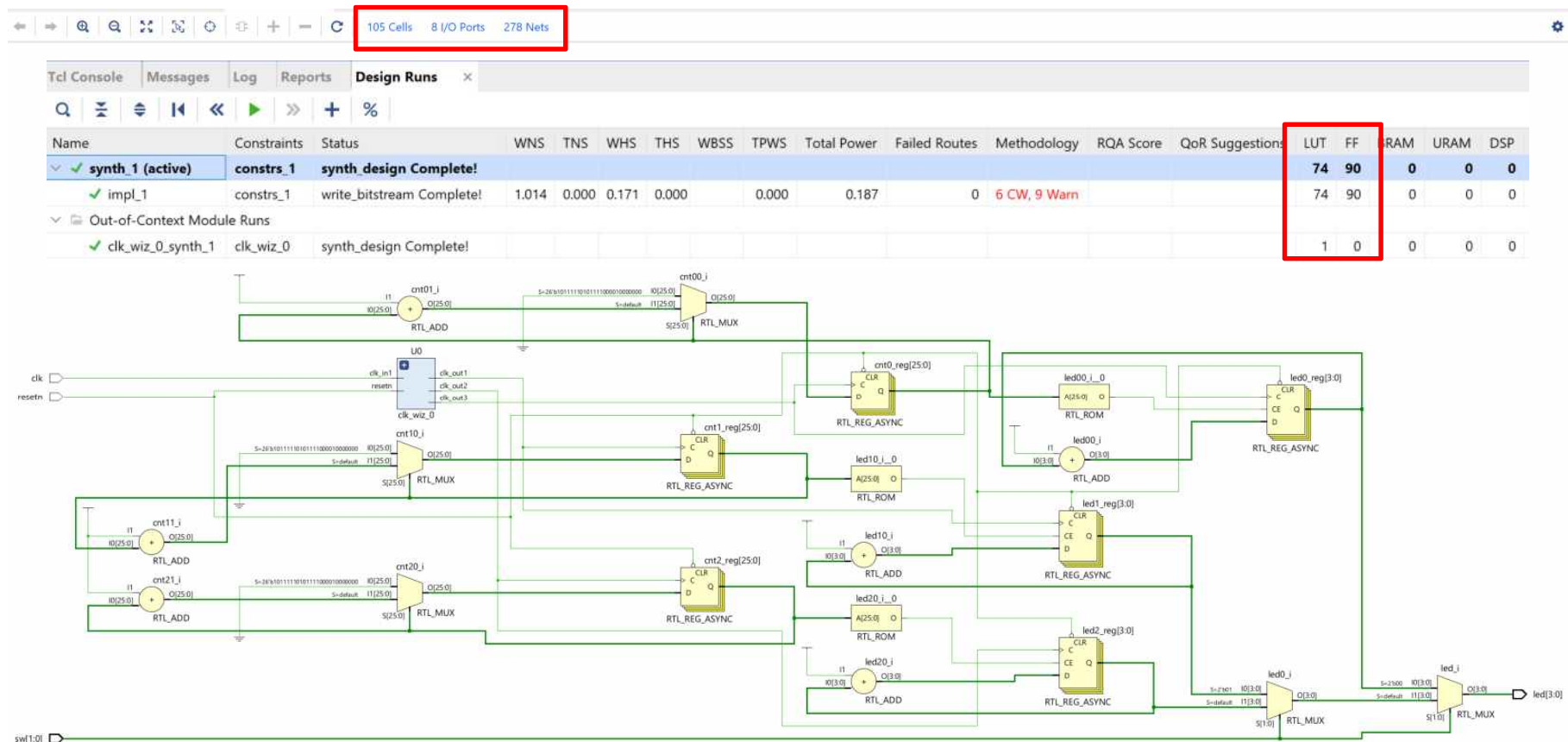
➤ Clock IP Test ➔ Code Implementation

✓ Clock_counter.v ➔ Design Source 생성 ➔ led on/off 주기 변경 구현

- [1:0] sw 의 값에 따라 Led ➔ On / Off 주기 변경
- [1:0] sw ➔ 2'b00 ➔ 200Mhz 로 동작 (LD0 : 0.25초 ~ LD3 : 2초)
- [1:0] sw ➔ 2'b01 ➔ 100Mhz 로 동작 (LD0 : 0.5초 ~ LD3 : 4초)
- [1:0] sw ➔ 2'b10 ➔ 50Mhz 로 동작 (LD0 : 1초 ~ LD3 : 8초)



Code Case I



Using Xilinx IP : Create Clock

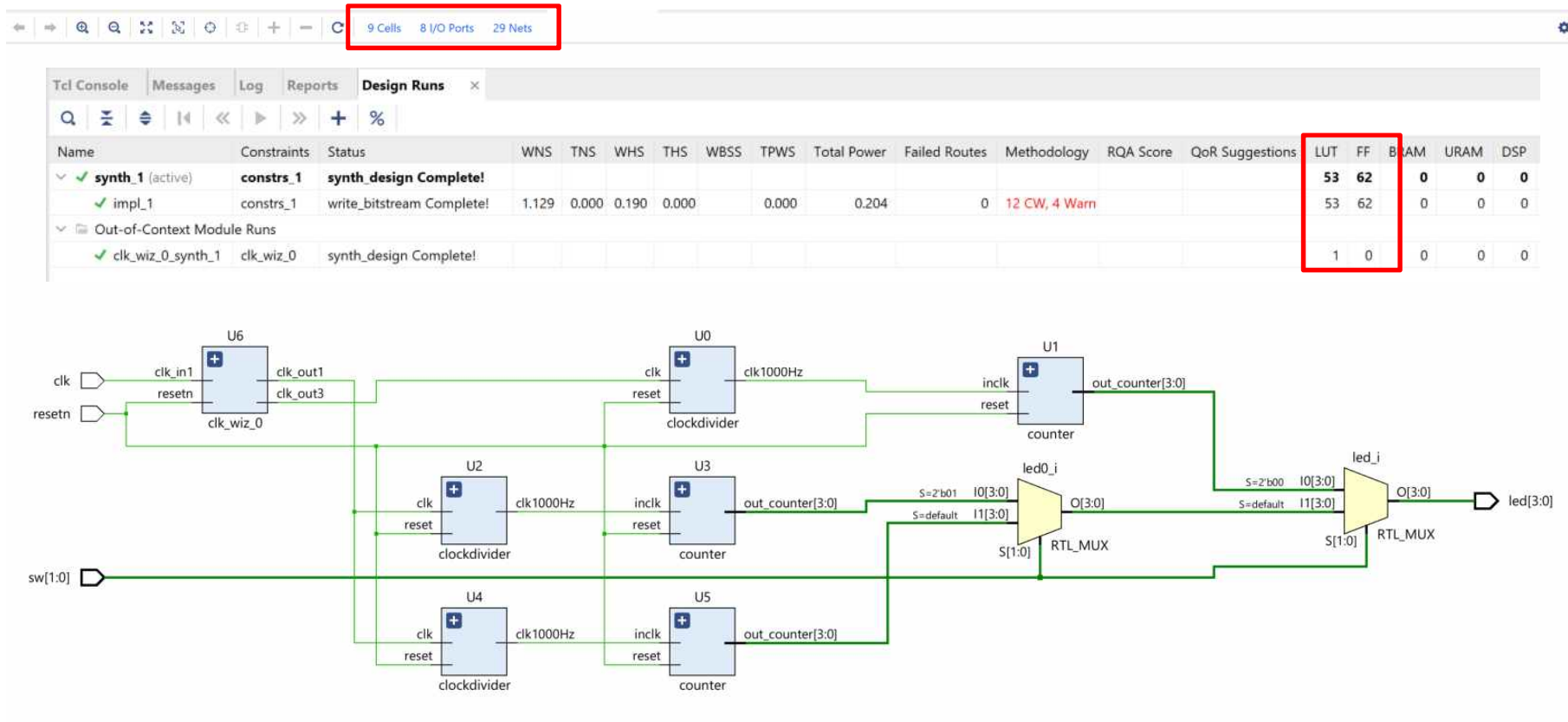
Create_Clock.xpr
Create_Clock_Exam.xpr

➤ Clock IP Test ➔ Code Implementation

✓ Clock_counter.v ➔ Design Source 생성 ➔ led on/off 주기 변경 구현

- [1:0] sw 의 값에 따라 Led ➔ On / Off 주기 변경
- [1:0] sw ➔ 2'b00 ➔ 200Mhz 로 동작 (LD0 : 0.25초 ~ LD3 : 2초)
- [1:0] sw ➔ 2'b01 ➔ 100Mhz 로 동작 (LD0 : 0.5초 ~ LD3 : 4초)
- [1:0] sw ➔ 2'b10 ➔ 50Mhz 로 동작 (LD0 : 1초 ~ LD3 : 8초)

❖ Code Case II



Using Xilinx IP : Create Clock

Create_Clock.xpr
Create_Clock_Exam.xpr

➤ Clock IP Test → Code Implementation

✓ Clock_counter.v → Design Source 생성 → led on/off 주기 변경 구현

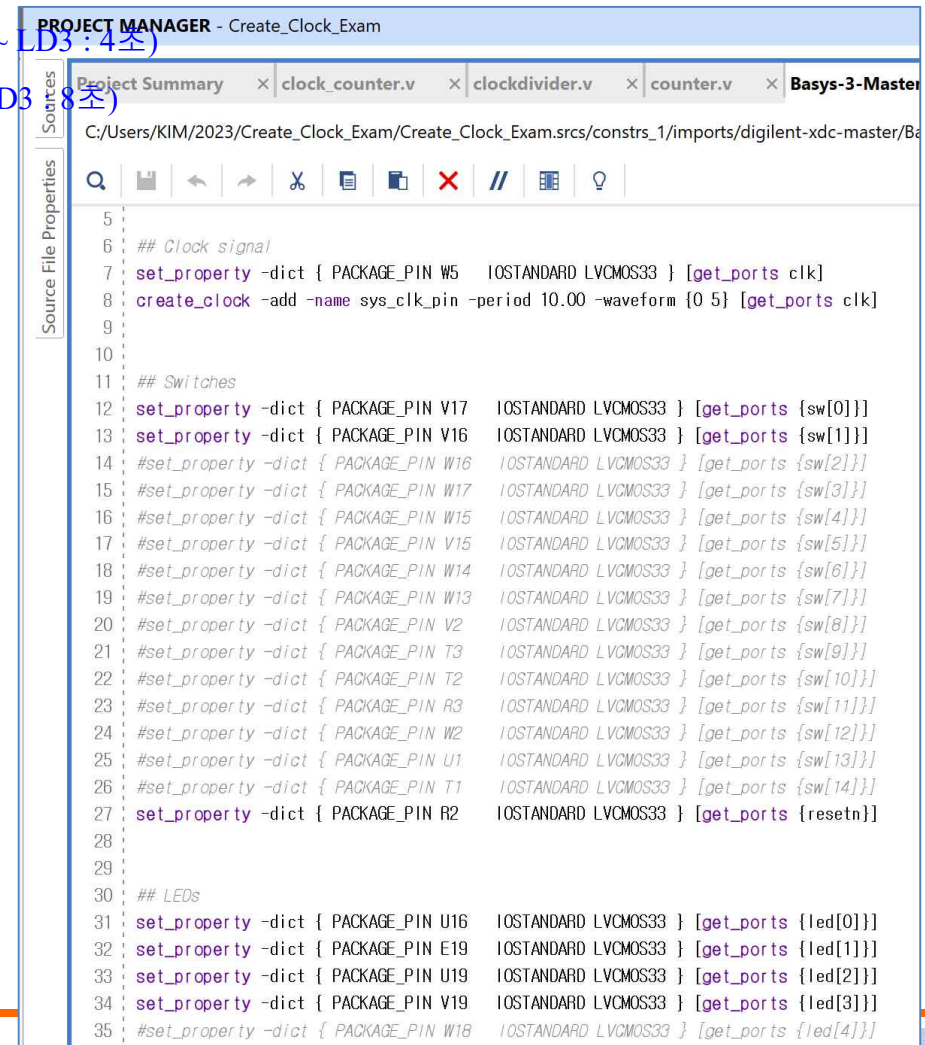
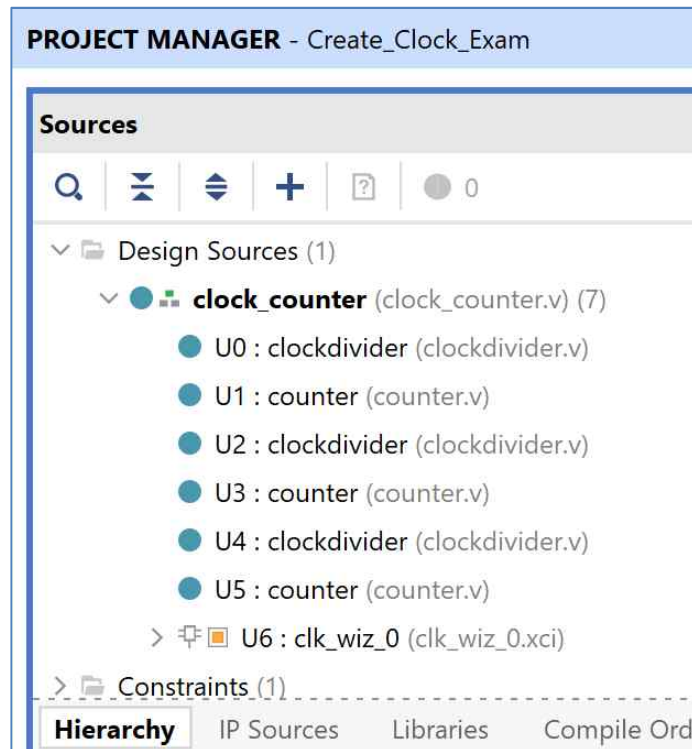
- [1:0] sw 의 값에 따라 Led → On / Off 주기 변경
- [1:0] sw → 2'b00 → 200Mhz 로 동작 (LD0 : 0.25초 ~ LD3 : 2초)
- [1:0] sw → 2'b01 → 100Mhz 로 동작 (LD0 : 0.5초 ~ LD3 : 4초)
- [1:0] sw → 2'b10 → 50Mhz 로 동작 (LD0 : 1초 ~ LD3 : 8초)



Code Case I



Code Case II



Code Case II

Using Xilinx IP : Create Clock

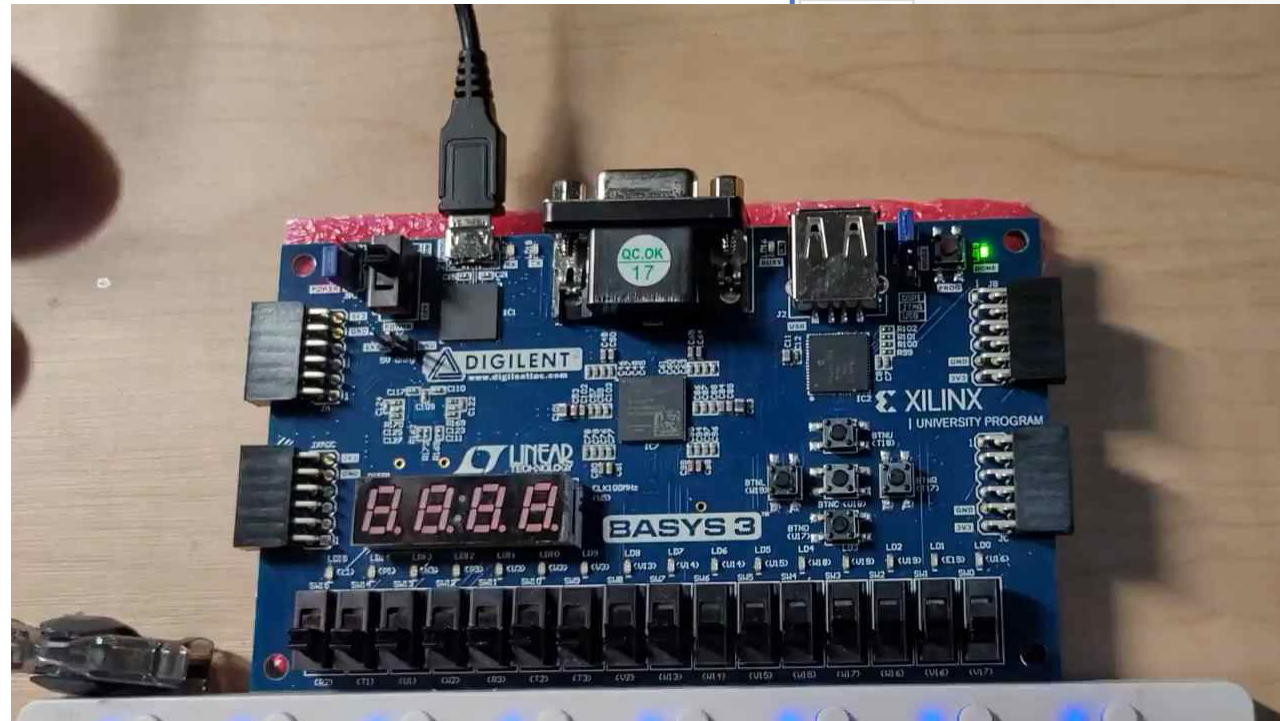
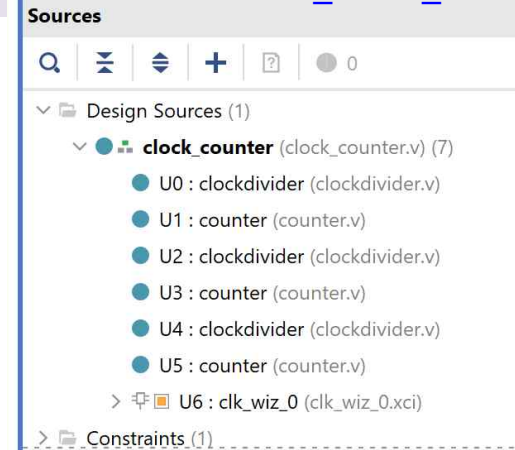
➤ Clock IP Test ➔ Code Implementation

✓ Clock_counter.v ➔ Design Source 생성 ➔ led on/off 주기 변경 구현

- [1:0] sw 의 값에 따라 Led ➔ On / Off 주기 변경
- [1:0] sw ➔ 2'b00 ➔ 200Mhz 로 동작 (LD0 : 0.25초 ~ LD3 : 2초)
- [1:0] sw ➔ 2'b01 ➔ 100Mhz 로 동작 (LD0 : 0.5초 ~ LD3 : 4초)
- [1:0] sw ➔ 2'b10 ➔ 50Mhz 로 동작 (LD0 : 1초 ~ LD3 : 8초)

- *clk* ➔ *clk*
- *SW0* ➔ *sw[0]*
- *SW1* ➔ *sw[1]*
- *reset* ➔ *SW15*
- *LD0* ➔ *led[0]*
- *LD1* ➔ *led[1]*
- *LD2* ➔ *led[2]*
- *LD3* ➔ *led[3]*

PROJECT MANAGER - Create_Clock_Exam
Create_Clock_Exam .xpr



❖ Code Case I ❖ Code Case II

Using Xilinx IP

- ✓ *Create Clock IP*
- ✓ *Code Implementation for Clock IP Test (Case I)*
- ✓ *Code Implementation for Clock IP Test (Case II)*
- ✓ *Create Memory IP*

Using Xilinx IP : Create Memory
[Memory Test (1) ➔ Single Port RAM]

Using Xilinx IP : Create Memory

Create_BlockMemory .xpr

➤ Create Clock

- Clocking Wizard를 통하여 자신이 원하는 Clock 을 생성 가능
- Clock generation → clock을 이용하여 주파수(Frequency)와 듀티(Duty), 페이즈(Phase)를 원하는 값으로 변경 [출력 주파수 = 입력주파수 * N / M(분주비)]

➤ Create Memory

- Memory 를 생성하는 IP : FPGA 내부의 Memory (**Block Memory(BM)**), Distributed Memory(DM))를 사용하여 Memory Block을 생성하는 IP
 - **Block Memory Generator(BMG)**는 내부 Block Memory 를 사용하여 RAM, ROM 을 쉽게 생성
- Memory Interface 를 생성하는 IP : DDR 과 같은 외부 Memory 와의 Interface를 생성하는 IP

Using Xilinx IP : Create Memory

Create_BlockMemory .xpr

➤ Create Clock

- Clocking Wizard를 통하여 자신이 원하는 Clock 을 생성 가능
- Clock generation → clock을 이용하여 주파수(Frequency)와 듀티(Duty), 페이즈(Phase)를 원하는 값으로 변경 [출력 주파수 = 입력주파수 * N / M(분주비)]

➤ Create Memory

- Memory 를 생성하는 IP : FPGA 내부의 Memory (**Block Memory(BM)**), Distributed Memory(DM))를 사용하여 Memory Block을 생성하는 IP
 - **Block Memory Generator(BMG)**는 내부 Block Memory 를 사용하여 RAM, ROM 을 쉽게 생성
- Memory Interface 를 생성하는 IP : DDR 과 같은 외부 Memory 와의 Interface를 생성하는 IP

Artix-7 FPGA Feature Summary

Table 4: Artix-7 FPGA Feature Summary by Device

Device	Logic Cells	Configurable Logic Blocks (CLBs)		DSP48E1 Slices ⁽²⁾	Block RAM Blocks ⁽³⁾			CMTs ⁽⁴⁾	PCIe ⁽⁵⁾	GTPs	XADC Blocks	Total I/O Banks ⁽⁶⁾	Max User I/O ⁽⁷⁾
		Slices ⁽¹⁾	Max Distributed RAM (Kb)		18 Kb	36 Kb	Max (Kb)						
XC7A12T	12,800	2,000	171	40	40	20	720	3	1	2	1	3	150
XC7A15T	16,640	2,600	200	45	50	25	900	5	1	4	1	5	250
XC7A25T	23,360	3,650	313	80	90	45	1,620	3	1	4	1	3	150
XC7A35T	33,280	5,200	400	90	100	50	1,800	5	1	4	1	5	250
XC7A50T	52,160	8,150	600	120	150	75	2,700	5	1	4	1	5	250
XC7A75T	75,520	11,800	892	180	210	105	3,780	6	1	8	1	6	300
XC7A100T	101,440	15,850	1,188	240	270	135	4,860	6	1	8	1	6	300
XC7A200T	215,360	33,650	2,888	740	730	365	13,140	10	1	16	1	10	500

Notes:

- Each 7 series FPGA slice contains four LUTs and eight flip-flops; only some slices can use their LUTs as distributed RAM or SRLs.
- Each DSP slice contains a pre-adder, a 25 x 18 multiplier, an adder, and an accumulator.
- Block RAMs are fundamentally 36 Kb in size; each block can also be used as two independent 18 Kb blocks.
- Each CMT contains one MMCM and one PLL.
- Artix-7 FPGA interface blocks for PCI Express support up to x4 Gen 2.
- Does not include configuration Bank 0.
- This number does not include GTP transceivers.

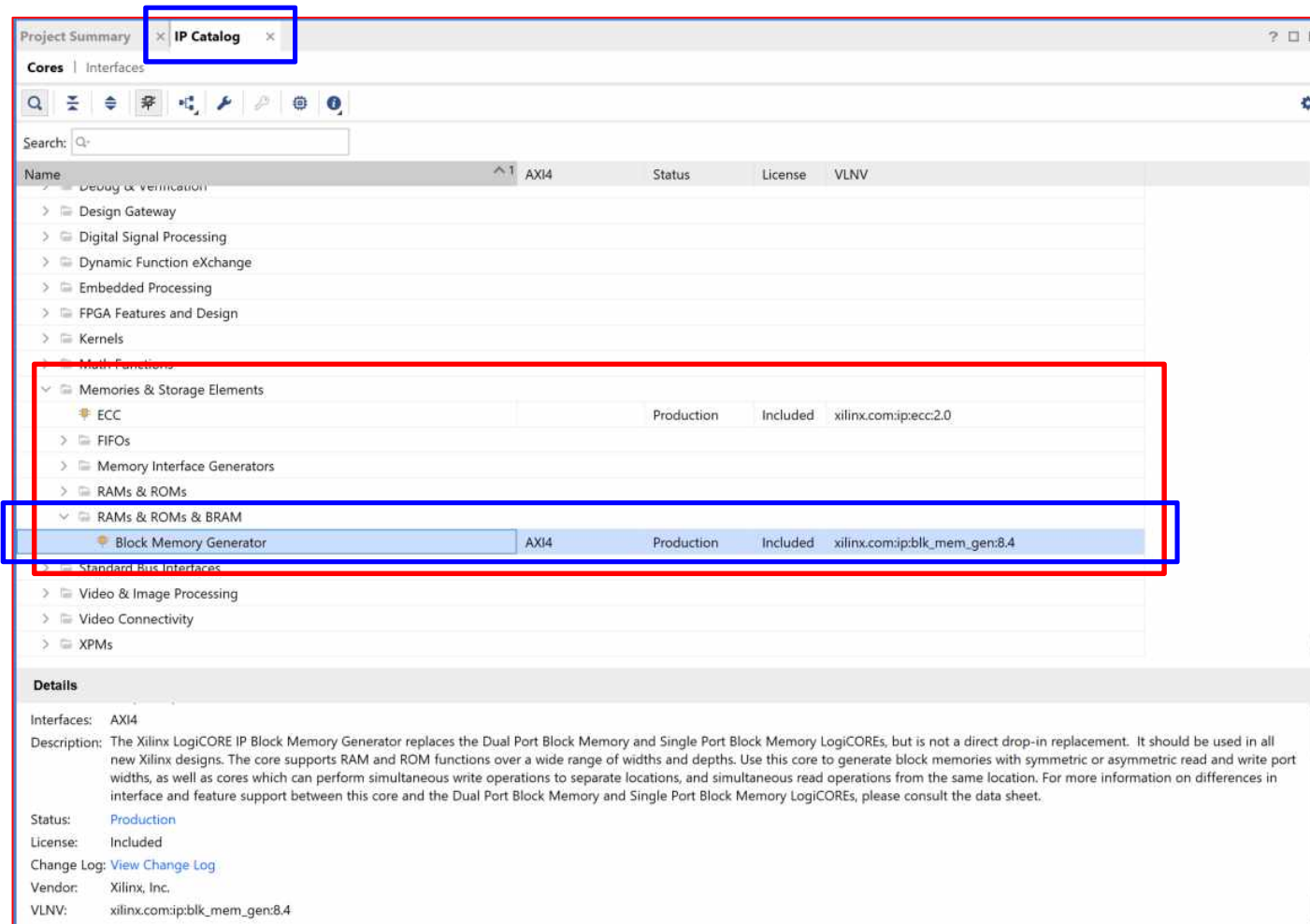
Using Xilinx IP

Create_BlockMemory .xpr

➤ Create Memory → Block Memory Generator(BMG)

① IP Catalog → Memories & Storage Elements → RAMs & ROMs & BRAM → **Block Memory Generator**: □

블 클릭



Using Xilinx IP

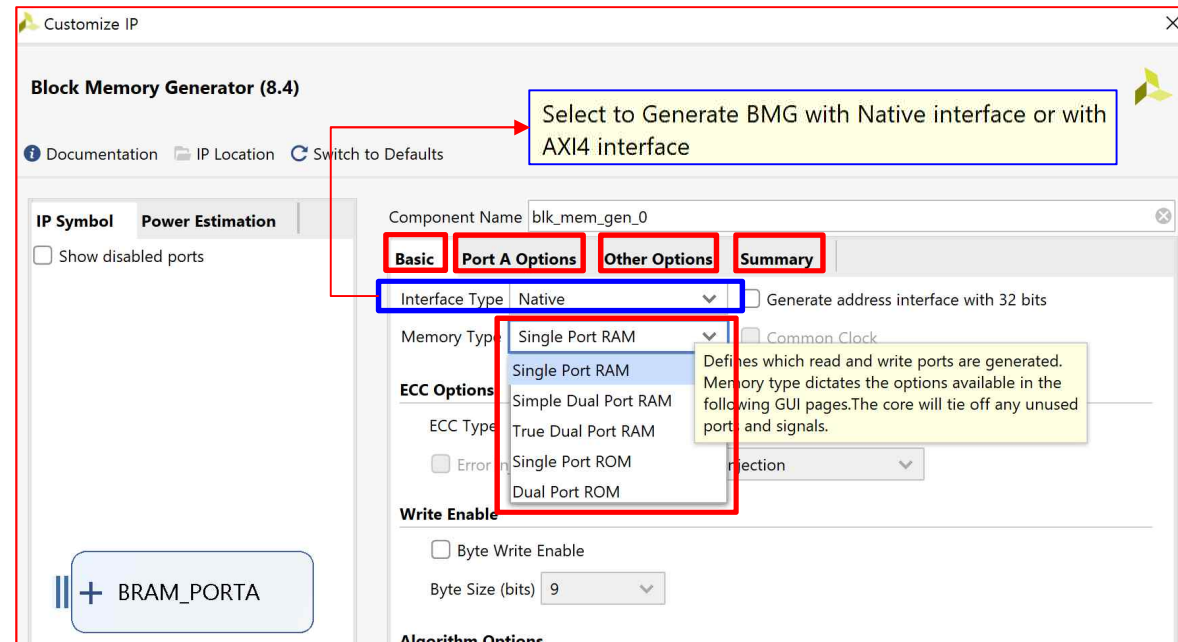
Create_BlockMemory .xpr

➤ Create Memory → Block Memory

Generator(BMG)

- ① IP Catalog → Memories & Storage
Elements → RAMs & ROMs &
BRAM → Block Memory
Generator : 더블 클릭

- ② **Block Memory Generator(BMG)**



▪ Interface Type : Native or AXI4

- **Native** : 사용자가 직접 Interface를 구현
- **AXI4** : Microblaze, Zynq 와 같은 Process와의 Interface에서 사용

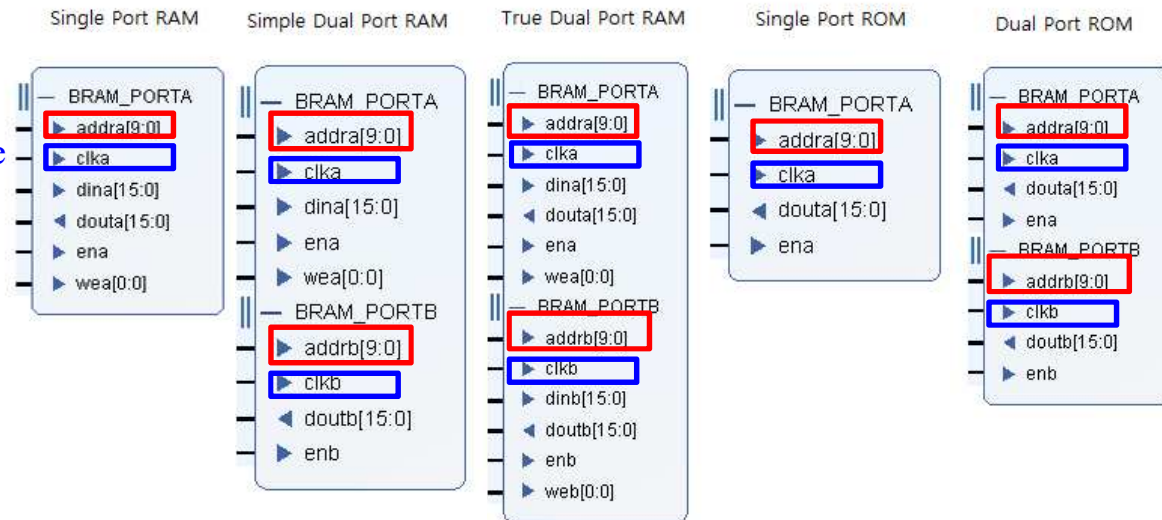
▪ Memory Type

1. Single Port RAM(**SPRAM**) : [PortA → write / read] → [단일 clock 사용]
2. Simple Dual Port RAM(**SDPRAM**) : [PortA → write], [PortB → read] → [dual clock 사용]
3. True Dual Port RAM(TDPRAM) : [PortA → read/write], [Port → read/write] → [dual clock 사용]
4. Single Port ROM : [PortA → read] → [단일 clock 사용]
5. Dual Port ROM : [PortA → read], [PortB → Read] → [dual clock 사용]

➤ Create Memory → Block Memory

Generator(BMG)

- ① IP Catalog → Memories & Storage
Elements → RAMs & ROMs &
BRAM → Block Memory
Generator : 더블 클릭
- ② Block Memory Generator(BMG)
- ③ **Memory Type 에 따른 I/O Port**



- Memory Type을 선택할 때 주요 사항
 - ➔ Single Port를 사용할지, Dual Port를 사용할지 (**하나의 Module에서 Access 할 때에는 Single Port, 두개의 Module에서 Access 할 때에는 Dual Port를 사용**)를 고려
 - ➔ Single Clock을 사용할지, Dual Clock을 사용할지 (read, write clock이 다른지, Access하는 두개의 Module에서 사용하는 clock이 다른지 등을 고려)를 고려해서 적당한 Memory를 선택
- [실습] **Single Port RAM** 과 **Simple Dual Port RAM**을 직접 생성(**Generation**)하고, **Simulation**을 통하여 동작의 특성을 확인

Using Xilinx IP

Create_BlockMemory .xpr

➤ Create Memory → Block Memory Generator(BMG)

① Block Memory Generator(BMG)

② → **Single Port RAM** → [Component Name : spram1] → [Port Option] → Memory의 Data Width, Depth(Size)

를 설정

Customize IP

Block Memory Generator (8.4)

Documentation IP Location Switch to Defaults

IP Symbol Power Estimation

Show disabled ports

Component Name blk_mem_gen_0

Component Name : *spram1*

Basic Port A Options Other Options Summary

Memory Size

Write Width 16 Range: 1 to 4608 (bits)

Read Width 16

Write Depth 1024 Range: 2 to 1048576

Read Depth 1024

Operating Mode Write First Enable Port Type Use ENA Pin

Port A Options

Write First

Read First

No Change

WRITE_FIRST: This mode is recommended when asynchronous clocks might cause simultaneous read/write operations on the same port address

NO_CHANGE: This mode ensures Lowest power however does not guarantee no collisions when both ports access same address at the same clock cycle

READ_FIRST: This mode guarantees no collisions (read will access prior memory contents safely) at the cost of higher BRAM power

Port A Output Reset Options

RSTA Pin (set/reset pin)

Reset Memory Latch

Reset Priority CE (Latch or Register Enable)

READ Address Change A

Read Address Change A

OK Cancel

+ BRAM_PORTA

Next Slide

Using Xilinx IP

Create_BlockMemory .xpr

➤ Create Memory ➔ Block Memory Generator(BMG) ➔ Memory Test (1) ➔ Single Port RAM

|| + BRAM_PORTA

① Memory Generation ➔ Simulation Result (Test Bench)

- ✓ Single Port RAM은 단일 Clock을 사용하고, 단일 Address를 사용
- ✓ wea 값에 의해 read/write 설정 ➔ wea가 1이면 write, wea가 0이면 read
- ✓ Operating Mode : Write First, Read First, No Change
 - ➔ din을 통하여 데이터를 write 할 때, dout에 어떤 데이터가 나오는지를 결정

➤ Write First Mode

- read, write를 동시에 하지 않는 경우에 사용
- din을 통하여 데이터를 write 할 때, dout 값에 din 값이 delay
 - ➔ dout에는 din으로 입력된 데이터가 2-clock 후에 나타남(2-Clock Delay)

➤ Read First Mode

- write 전에 이전의 write 되었던 값을 read 해서 사용해야 하는 경우에 사용
- din을 통하여 데이터를 write 할 때, dout 값에 이전에 저장되었던 값이 delay
 - ➔ dout에는 din으로 입력된 데이터가 2-clock 후에 나타남
- 주로 image data를 라인 단위로 저장 후, 처리하는 Line Memory 구현의 경우 사용

➤ No Change Mode

- din을 통하여 데이터를 write 할 때, dout 값에 이전의 dout 값이 계속해서 나타남
- read, write ➔ 독립적으로 동작
- dout, din ➔ 독립적으로 동작

Continue

- Interface Type : Native or AXI4

- Memory Type

1. Single Port RAM(SPRAM)
 - : PortA ➔ write / read
2. Simple Dual Port Ram(SDPRAM)
 - : PortA ➔ write, PortB ➔ read
3. True Dual Port RAM(TDPRAM)
 - : PortA는 write, PortB는 read
4. Single Port ROM
 - : PortA ➔ Read
5. Dual Port ROM
 - : PortA & PortB ➔ Read

- Create Memory ➔ Block Memory Generator(BMG) ➔ Memory Test (1) ➔ **Single Port RAM : 유의 사항**
- ① Memory Generation ➔ **Simulation Result (Test Bench)**

✓ 메모리 생성시 유의 사항 : **REGCEA(or REGCEB) Pin**

- 이 핀을 Check 해서 메모리 생성시 이 핀을 사용 ➔ 이 핀에 1을 입력해 주어야만 read 과정에서 발생할 수 있는 오류를 방지
- **REGCEA(B) 핀이 없으면 read 하려고 한 개수만큼 read 되지 않는 오류가 발생 ➔**
 - ❖ 예를 들면 1024개의 data를 연속해서 read 하려고 할 때, 1023개의 데이터만 read 되고, 마지막 1024번째 데이터는 read 되지 않는 오류
 - ❖ ena 신호가 항상 Enable (1)인 경우에는 이러한 문제가 발생하지 않지만, ena 신호가 read하려는 구간에서만 Enable(1)인 경우에는 이러한 오류가 발생
 - ❖ 따라서 사용자는 습관적으로 메모리 생성시 **REGCEA(B) Pin 항목을 Check해서 메모리를 생성하고 이 핀에 “1”을 입력 ➔ Single Port, Simple Dual Port, True Dual Port RAM에 모두 해당**

✓ Simulation 을 통해 read 오류 확인 및 검증

➤ Create Memory → Block Memory Generator(BMG)

- ① Block Memory Generator(BMG) → Single Port RAM → [Port Option] → Memory의 Data Width, Depth(Size)를 설정
- ② Others Options → **Memory Initialization**: 메모리의 초기값을 설정하며, ROM을 구현할 때, ROM의 초기 데이터를 설정하는 용도로 사용

Component Name spram1

Basic Port A Options **Other Options** Summary

Pipeline Stages within Mux 0 Mux Size: 1x1

Memory Initialization

☐ Load Init File

Coe File no_coe_file_loaded Browse Edit

☐ Fill Remaining Memory Locations

Remaining Memory Locations (Hex) 0

Structural/UniSim Simulation Model Options

Defines the type of warnings and outputs are generated when a read-write or write-write collision occurs.

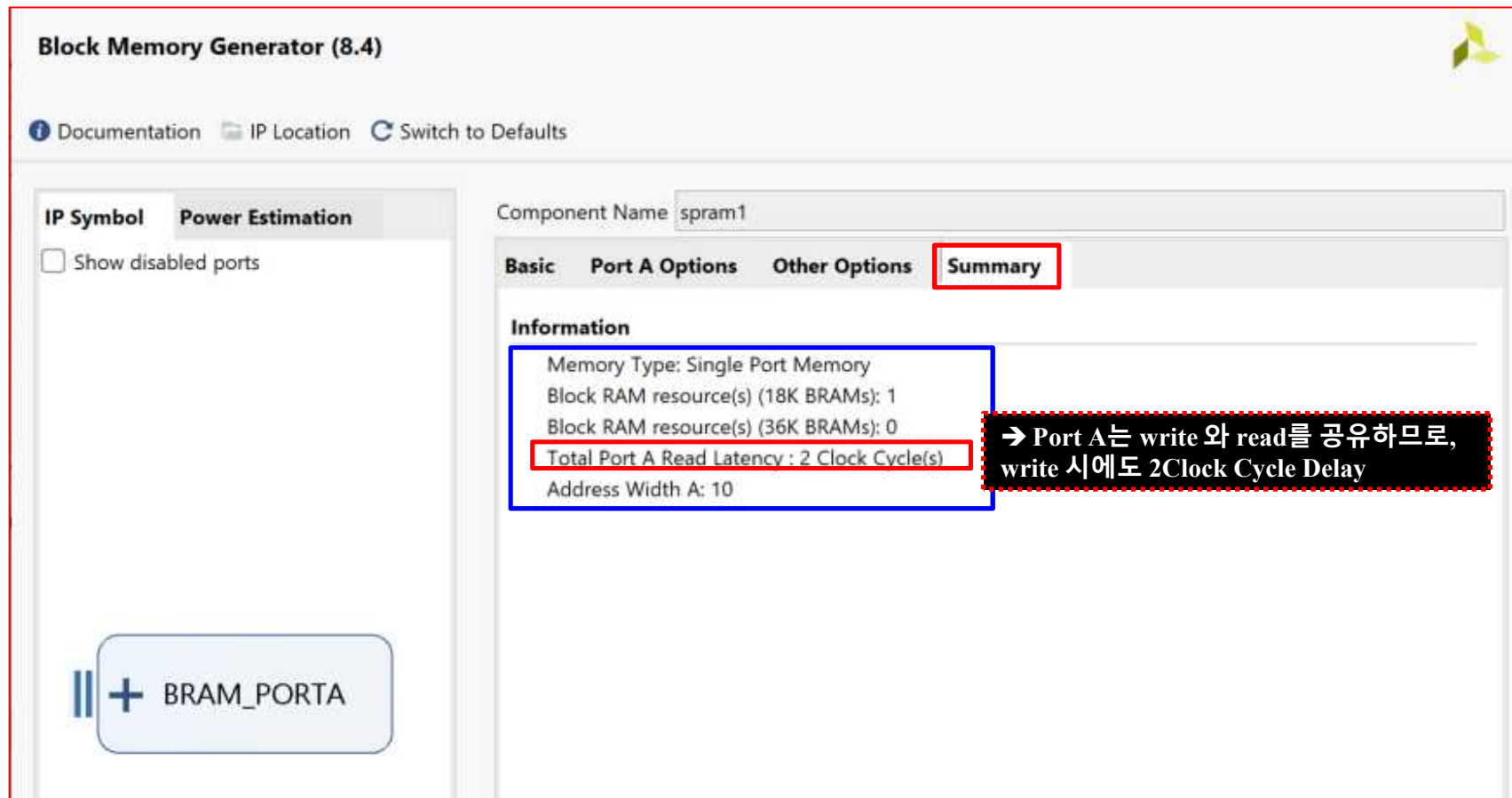
Collision Warnings All

Behavioral Simulation Model Options

☐ Disable Collision Warnings ☐ Disable Out of Range Warnings

➤ Create Memory → Block Memory Generator(BMG)

- ① Others Options → Memory Initialization : 메모리의 초기값을 설정하며, ROM을 구현할 때, ROM의 초기 데이터를 설정하는 용도로 사용
- ② Memory의 **Summary** : 사용된 **Memory Resource**와 설정된 값 [**Read Latency : 2 Clock Cycle(s)** → Read를 할 때, dout 데이터가 2-clock delay] : FPGA 설계에서 중요한 용어 중에 하나가 **Latency, Delay, Clock** [주의]



Using Xilinx IP

Create_BlockMemory .xpr

➤ Create Memory → Block Memory Generator(BMG) → **Memory Test (1) → Single Port RAM : spram1**

① Memory Generation → Simulation Result (Test Bench)

② IP Catalog → Memories & Storage Elements – RAMs & ROMs & BRAM – **Block Memory Generator** 더블

클릭 → **6개의 메모리 생성(1)**

1) spram1

- ✓ Component Name : **spram1**
- ✓ Memory Type : **Single Port RAM**
- ✓ Write Width, Read Width : **16**
- ✓ Write Depth, Read Depth : **1024**
- ✓ Operating Mode : **Write First**
- ✓ REGCEA Pin : **UnCheck**

Defines whether the configured memory has a register enabled pin (REGCEA). If there is no register enabled pin, the output register is enabled using the enable signal itself.

Using Xilinx IP

Create_BlockMemory .xpr

➤ Create Memory → Block Memory Generator(BMG) → Memory Test (1) → Single Port RAM : spram2

① IP Catalog → Memories & Storage Elements – RAMs & ROMs & BRAM – Block Memory Generator 더블 클릭 → 6개의 메모리 생성(2)

2) spram2

- ✓ Component Name : **spram2**
- ✓ Memory Type : Single Port RAM
- ✓ Write Width, Read Width : 16
- ✓ Write Depth, Read Depth : 1024
- ✓ Operating Mode : **Read First**
- ✓ REGCEA Pin : **UnCheck**

Defines whether the configured memory has a register enabled pin (REGCEA). If there is no register enabled pin, the output register is enabled using the enable signal itself.

Using Xilinx IP

Create_BlockMemory .xpr

➤ Create Memory → Block Memory Generator(BMG) → Memory Test (1) → Single Port RAM : spram3

① IP Catalog → Memories & Storage Elements – RAMs & ROMs & BRAM – Block Memory Generator 더블
클릭 → 6개의 메모리 생성(3)

3) spram3

- ✓ Component Name : **spram3**
- ✓ Memory Type : Single Port RAM
- ✓ Write Width, Read Width : 16
- ✓ Write Depth, Read Depth : 1024
- ✓ Operating Mode : **No Change**
- ✓ REGCEA Pin : **UnCheck**

Defines whether the configured memory has a register enabled pin (REGCEA). If there is no register enabled pin, the output register is enabled using the enable signal itself.

Using Xilinx IP

Create_BlockMemory .xpr

➤ Create Memory → Block Memory Generator(BMG) → Memory Test (1) → Single Port RAM : spram4

① IP Catalog → Memories & Storage Elements – RAMs & ROMs & BRAM – Block Memory Generator 더블
클릭 → 6개의 메모리 생성(4)

PROJECT MANAGER

- Settings
- Add Sources
- Language Templates
- IP Catalog

Block Memory Generator (8.4)

Documentation IP Location Switch to Defaults

IP Symbol Power Estimation

Show disabled ports

Basic Port A Options Other Options Summary

Interface Type Native General

Memory Type Single Port RAM

Component Name spram4

Memory Size

Write Width 16 Range: 1 to 4608 (bits)

Read Width 16

Write Depth 1024 Range: 2 to 1048576

Read Depth 1024

Operating Mode Write First Enable Port Type Use ENA Pin

Port A Optional Output Registers

☒ Primitives Output Register ☐ Core Output Register

☐ SoftECC Input Register ☒ REGCEA Pin

BRAM_PORTA regcea

Defines whether the configured memory has a register enabled pin (REGCEA). If there is no register enabled pin, the output register is enabled using the enable signal itself.

OK Cancel

4) spram4

- ✓ Component Name : spram4
- ✓ Memory Type : Single Port RAM
- ✓ Write Width, Read Width : 16
- ✓ Write Depth, Read Depth : 1024
- ✓ Operating Mode : Write First
- ✓ REGCEA Pin : Check

Using Xilinx IP

Create_BlockMemory .xpr

➤ Create Memory → Block Memory Generator(BMG) → Memory Test (1) → Single Port RAM : spram5

① IP Catalog → Memories & Storage Elements – RAMs & ROMs & BRAM – Block Memory Generator 더블 클릭 → 6개의 메모리 생성(5)

The screenshot shows the 'Customize IP' window for the 'Block Memory Generator (8.4)'. The 'Basic' tab is selected. The 'Component Name' is 'spram5'. The 'Memory Size' section shows 'Write Width' and 'Read Width' set to 16, and 'Write Depth' and 'Read Depth' set to 1024. The 'Operating Mode' is 'Read First'. The 'Port A Optional Output Registers' section shows 'REGCEA Pin' checked. A yellow box highlights the 'REGCEA Pin' checkbox with the text: 'Defines whether the configured memory has a register enabled pin (REGCEA). If there is no register enabled pin, the output register is enabled using the enable signal itself.'

5) spram5

- ✓ Component Name : spram5
- ✓ Memory Type : Single Port RAM
- ✓ Write Width, Read Width : 16
- ✓ Write Depth, Read Depth : 1024
- ✓ Operating Mode : Read First
- ✓ REGCEA Pin : Check

Using Xilinx IP

Create_BlockMemory .xpr

➤ Create Memory → Block Memory Generator(BMG) → Memory Test (1) → Single Port RAM : spram6

① IP Catalog → Memories & Storage Elements – RAMs & ROMs & BRAM – Block Memory Generator 더블
클릭 → 6개의 메모리 생성(6)

6) spram6

- ✓ Component Name : **spram6**
- ✓ Memory Type : Single Port RAM
- ✓ Write Width, Read Width : 16
- ✓ Write Depth, Read Depth : 1024
- ✓ Operating Mode : **No Change**
- ✓ REGCEA Pin : **Check**

Defines whether the configured memory has a register enabled pin (REGCEA). If there is no register enabled pin, the output register is enabled using the enable signal itself.

Using Xilinx IP : Create Memory
[*Memory Test (1)* → *Single Port RAM*]

→ *Test Bench*

➤ Create Memory → Block Memory Generator(BMG) → Memory Test (1) → Single Port RAM : spram6 →→→

→→ Simulation (Test Bench) : spram_tb.v 생성

① 6개의 메모리 생성(6) → **Simulation (Test Bench) : spram_tb.v 생성**

✓ Memory Test 순서

- ① clock은 100Mhz 단일 clock을 사용
- ② 카운터 (counter → cnt) 를 생성 : 0 ~ 2047 → cnt : 11비트 → 2048
- ③ 첫번째 cnt의 범위가 1024 ~ 2047 일 때, 데이터를 순차적으로 write
- ④ 두번째 cnt의 범위가 1024 ~ 2047 일 때, 데이터를 순차적으로 read
- ⑤ 6개(spram1~spram6)의 출력(dout1 ~ dout6) 값 확인

Using Xilinx IP

Create_BlockMemory .xpr

➤ Create Memory → Block Memory Generator(BMG) → Memory Test (1) → Single Port RAM : spram6 →→→

→→ Simulation (Test Bench) : spram_tb.v 생성

① 6개의 메모리 생성(6) → Simulation (Test Bench) : spram_tb.v 생성

spram_tb(1/3)

1. `timescale 1ns / 1ps

2. module spram_tb();

3. reg reset, clock;

4. initial begin

5. reset = 0;

6. clock = 0;

7.

8. #10000 reset = 1;

9. end

10. always #5 clock = ~clock; // 100 Mhz

11. reg [10:0] cnt; → cnt : 11비트 → 0 ~ 2047

12. always @(posedge clock or negedge reset)

13. begin

14. if(~reset) cnt <= 11'b0;

15. else cnt <= cnt+1'b1;

16. end

▪ 3 ~ 10 : reset, clock(100Mhz) 생성

▪ 11~16 : cnt 생성 → 0 ~ 2047

```
Project Summary x sdpram1.veo x spram1.veo x
C:/Users/KIM/2023/Create_BlockMemory/Create_BlockMemory.ip_user_files/ip/spram1/spram1.veo

53 // core to be instantiated. Change the instance name and port connections
54 // (in parentheses) to your own signal names.
55
56 //----- Begin Cut here for INSTANTIATION Template -----// INST_TAG
57 spram1 your_instance_name (
58   .clka(clka), // input wire clka
59   .ena(ena), // input wire ena
60   .wea(wea), // input wire [0 : 0] wea
61   .addra(addra), // input wire [9 : 0] addra
62   .dina(dina), // input wire [15 : 0] dina
63   .douta(douta) // output wire [15 : 0] douta
64 );
65 // INST_TAG_END ----- End INSTANTIATION Template -----
66
67 // You must compile the wrapper file spram1.v when simulating
68 // the core, spram1. When compiling the wrapper file, be sure to
69 // reference the Verilog simulation library.
70
```

spram1.veo

```
Project Summary x sdpram1.veo x
C:/Users/KIM/2023/Create_BlockMemory/Create_BlockMemory.ip_user_files/ip/sdpram1/sdpram1.veo

53 // core to be instantiated. Change the instance name and port connections
54 // (in parentheses) to your own signal names.
55
56 //----- Begin Cut here for INSTANTIATION Template -----// INST_TAG
57 sdpram1 your_instance_name (
58   .clka(clka), // input wire clka
59   .ena(ena), // input wire ena
60   .wea(wea), // input wire [0 : 0] wea
61   .addra(addra), // input wire [9 : 0] addra
62   .dina(dina), // input wire [15 : 0] dina
63   .clkb(clkb), // input wire clkb
64   .enb(enb), // input wire enb
65   .regceb(regceb), // input wire regceb
66   .addrb(addrb), // input wire [9 : 0] addrb
67   .doutb(doutb) // output wire [15 : 0] doutb
68 );
69 // INST_TAG_END ----- End INSTANTIATION Template -----
70
```

sdpram1.veo

Using Xilinx IP

Create_BlockMemory .xpr

➤ Create Memory → Block Memory Generator(BMG) → Memory Test (1) → Single Port RAM : spram6 → → →

→ → Simulation (Test Bench) : spram_tb.v 생성



Using Xilinx IP

➤ Create Memory → Block Memory Generator(BMG) → Memory Test (1)

→ Single Port RAM : spram6 → → → → Simulation (Test Bench) : spram_tb.v 생성

spram_tb(3/3)

```
42. wire [15:0] dout1;
43. spram1 spram1_u1 (
44.     .clka(clock), .ena(ena), .wea(wea),
45.     .addra(addr), .dina(din), .douta(dout1)
46. );
```

```
46. wire [15:0] dout2;
47. spram2 spram2_u1 (
48.     .clka(clock), .ena(ena), .wea(wea),
49.     .addra(addr), .dina(din), .douta(dout2)
50. );
```

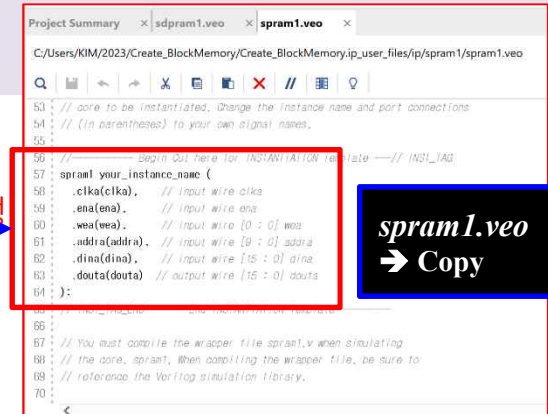
```
50. wire [15:0] dout3;
51. spram3 spram3_u1 (
52.     .clka(clock), .ena(ena), .wea(wea),
53.     .addra(addr), .dina(din), .douta(dout3)
54. );
```

```
54. wire [15:0] dout4;
55. spram4 spram4_u1 (
56.     .clka(clock), .ena(ena), regcea(1'b1), .wea(wea),
57.     .addra(addr), .dina(din), .douta(dout4)
58. );
```

```
58. wire [15:0] dout5;
59. spram5 spram5_u1 (
60.     .clka(clock), .ena(ena), regcea(1'b1), .wea(wea),
61.     .addra(addr), .dina(din), .douta(dout5)
62. );
```

```
62. wire [15:0] dout6;
63. spram6 spram6_u1 (
64.     .clka(clock), .ena(ena), regcea(1'b1), .wea(wea),
65.     .addra(addr), .dina(din), .douta(dout6)
66. );
```

66. endmodule



- 42~45 : spram1 모듈을 추가 → 출력값 : dout1
- 46~49 : spram2 모듈을 추가 → 출력값 : dout2
- 50~53 : spram3 모듈을 추가 → 출력값 : dout3
- 54~57 : spram4 모듈을 추가 → 출력값 : dout4
- 58~61 : spram5 모듈을 추가 → 출력값 : dout5
- 62~66 : spram6 모듈을 추가 → 출력값 : dout6
- spram1 ~ spram3 : regcea 포트가 없음
- spram4 ~ spram6 : regcea 포트에 1'b1 의 값

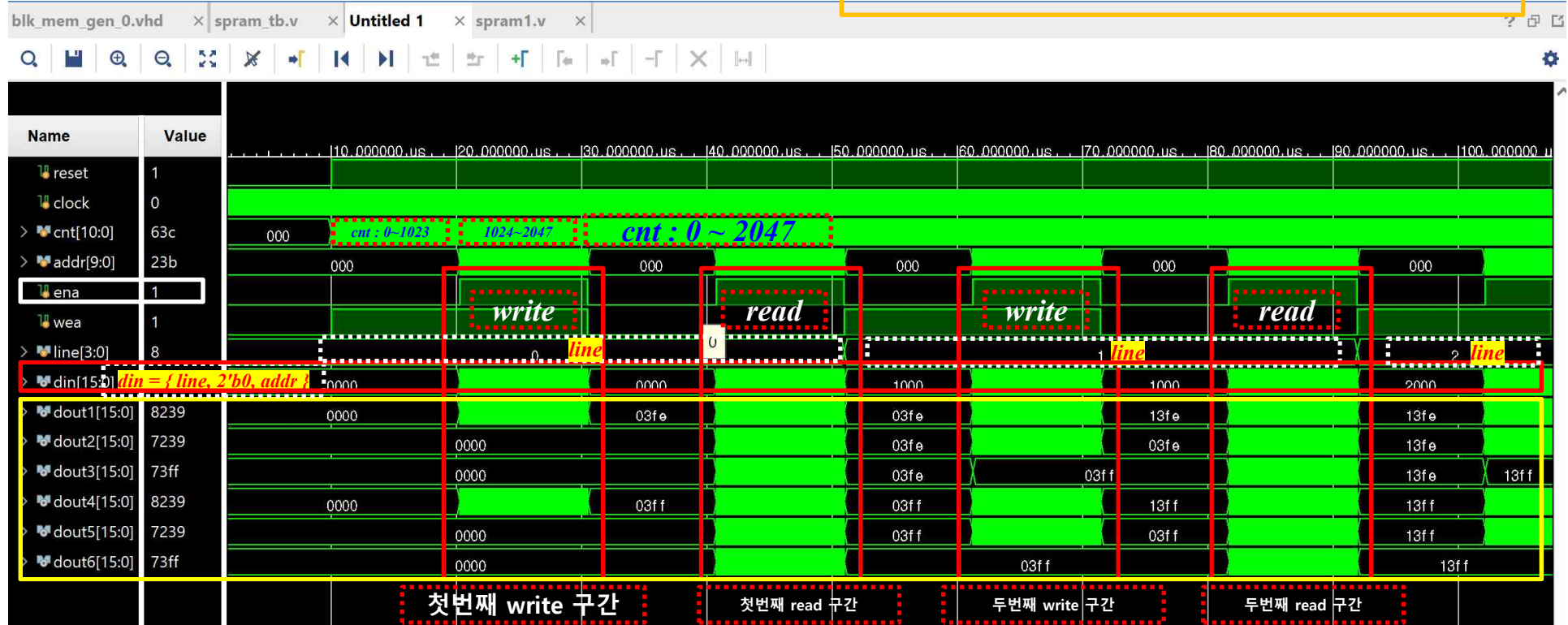
Using Xilinx IP

Create_BlockMemory .xpr

➤ Create Memory ➔ Block Memory Generator(BMG) ➔ Memory Test (1) ➔ Single Port RAM : spram6 ➔

Simulation (Test Bench) : spram_tb.v 생성 ➔ **Run Simulation**

▪ 1024개씩 data write, data read 가 반복



1) spram1

- ✓ Component Name : **spram1**
- ✓ Memory Type : Single Port RAM
- ✓ Write Width, Read Width : 16
- ✓ Write Depth, Read Depth : 1024
- ✓ Operating Mode : **Write First**
- ✓ REGCEA Pin : **UnCheck**

2) spram2

- ✓ Component Name : **spram2**
- ✓ Memory Type : Single Port RAM
- ✓ Write Width, Read Width : 16
- ✓ Write Depth, Read Depth : 1024
- ✓ Operating Mode : **Read First**
- ✓ REGCEA Pin : **UnCheck**

3) spram3

- ✓ Component Name : **spram3**
- ✓ Memory Type : Single Port RAM
- ✓ Write Width, Read Width : 16
- ✓ Write Depth, Read Depth : 1024
- ✓ Operating Mode : **No Change**
- ✓ REGCEA Pin : **UnCheck**

4) spram4

- ✓ Component Name : **spram4**
- ✓ Memory Type : Single Port RAM
- ✓ Write Width, Read Width : 16
- ✓ Write Depth, Read Depth : 1024
- ✓ Operating Mode : **Write First**
- ✓ REGCEA Pin : **Check**

5) spram5

- ✓ Component Name : **spram5**
- ✓ Memory Type : Single Port RAM
- ✓ Write Width, Read Width : 16
- ✓ Write Depth, Read Depth : 1024
- ✓ Operating Mode : **Read First**
- ✓ REGCEA Pin : **Check**

6) spram6

- ✓ Component Name : **spram6**
- ✓ Memory Type : Single Port RAM
- ✓ Write Width, Read Width : 16
- ✓ Write Depth, Read Depth : 1024
- ✓ Operating Mode : **No Change**
- ✓ REGCEA Pin : **Check**

Using Xilinx IP

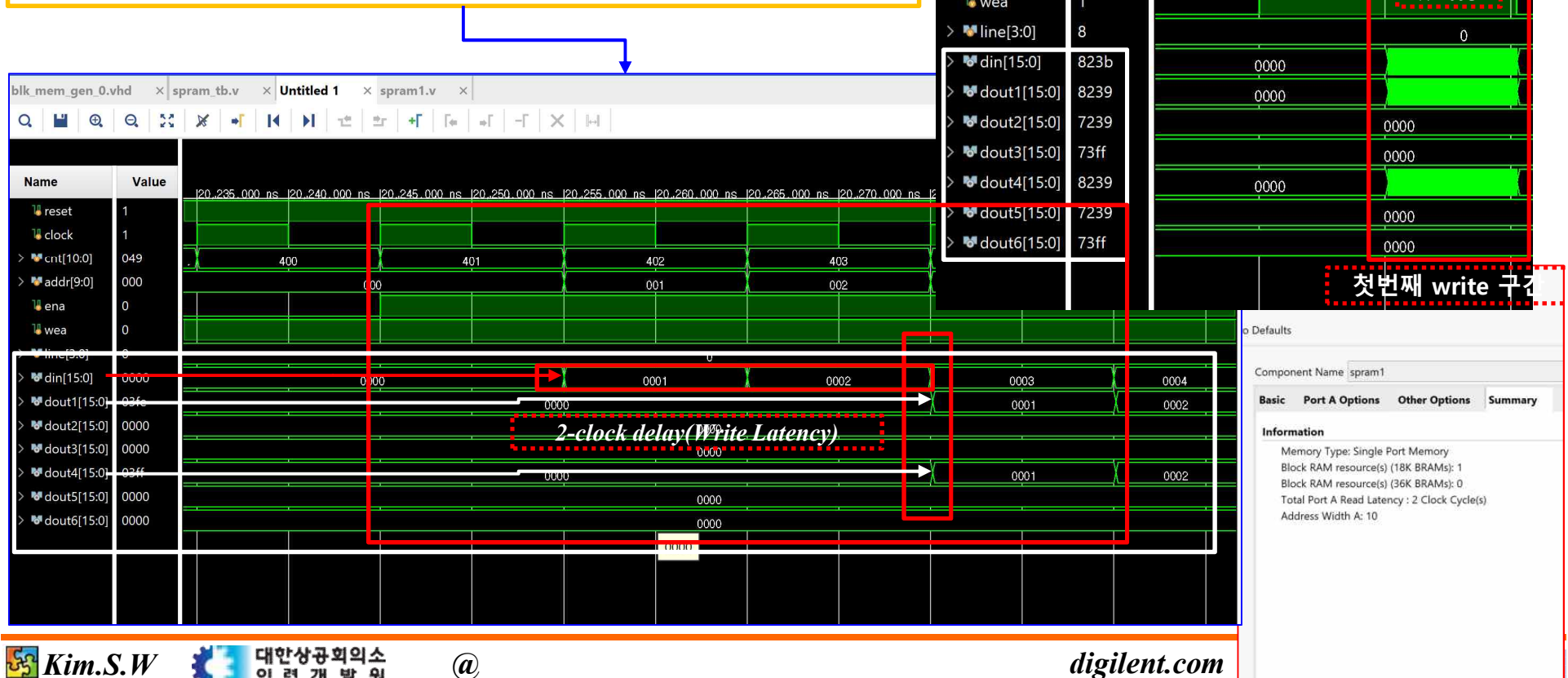
Create_BlockMemory .xpr

➤ Create Memory ➔ Block Memory Generator(BMG) ➔ Memory Test (1) ➔ Single Port RAM : spram6 ➔

Simulation (Test Bench) : spram_tb.v 생성 ➔ Run Simulation

▪ 첫번째 write 구간 시작부분

- ➔ dout1, dout4 : din의 2-clock delay 된 값(write first),
- ➔ dout2, dout5: [write 되었던 값 = 0] ➔ 0x000 (read first)
- ➔ dout3, dout6은 이전 값이 0이기 때문 [No change]에 계속해서 0x000 유지
- ➔ 2-clock delay (Write Latency, 쓰기 지연(대기)시간)



Using Xilinx IP

Create_BlockMemory .xpr

➤ Create Memory → Block Memory Generator(BMG) → Memory Test (1) → Single Port RAM : spram6 →

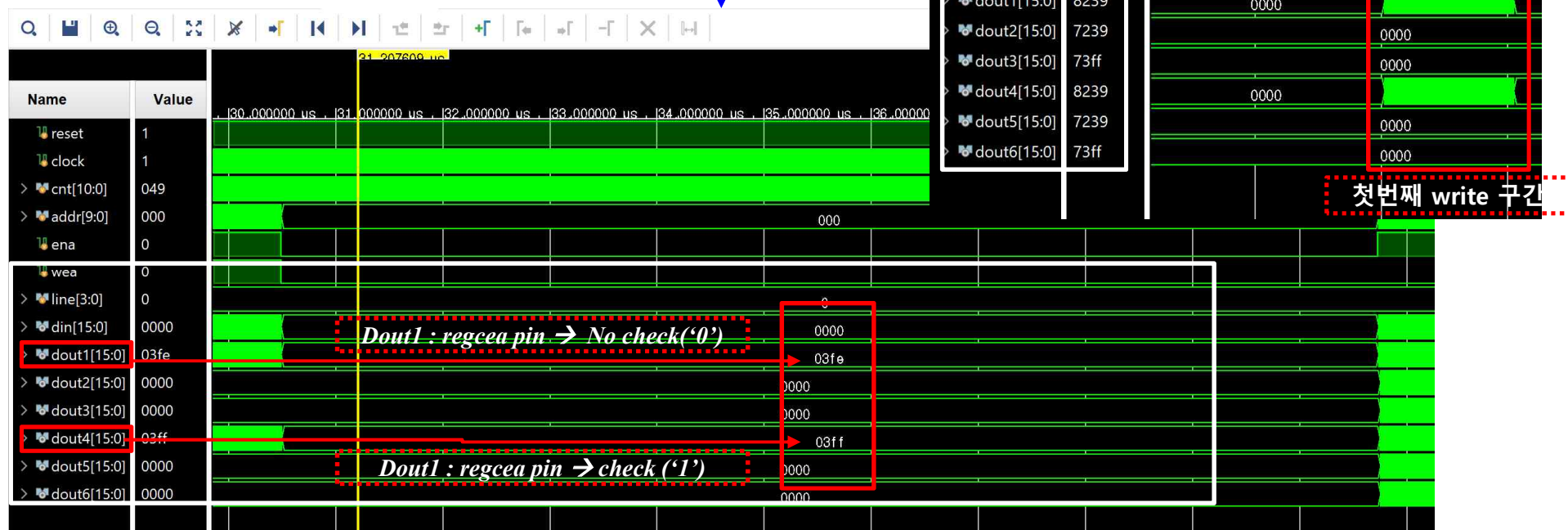
Simulation (Test Bench) : spram_tb.v 생성 → Run Simulation

▪ 첫번째 write 구간의 끝부분

→ dout1: regcea 포트('0')가 없는 모듈 → 0x03fe까지 data 출력

→ dout4는 regcea 포트에 "1"을 입력한 모듈 → 0x03ff 까지 1024개 data 모두 출력

→ 메모리 생성시 REGCEA Pin → 반드시 추가



Using Xilinx IP

Create_BlockMemory .xpr

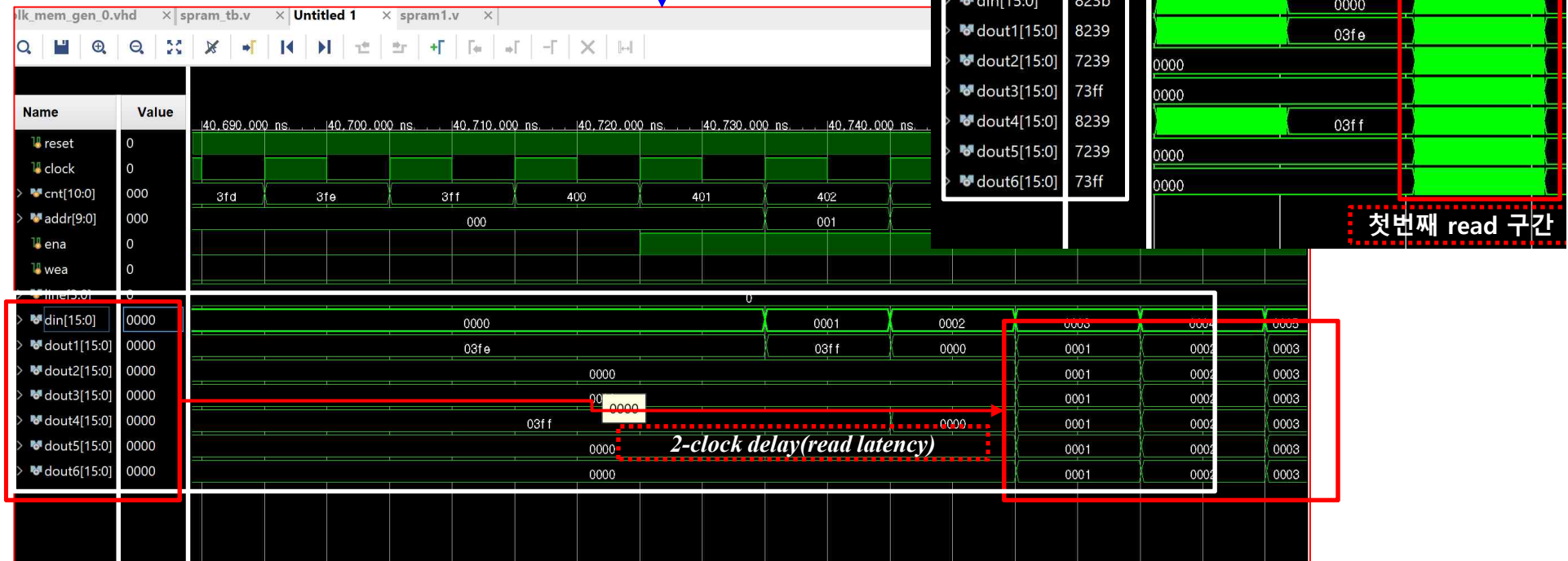
➤ Create Memory ➔ Block Memory Generator(BMG) ➔ Memory Test (1) ➔ Single Port RAM : spram6 ➔

Simulation (Test Bench) : spram_tb.v 생성 ➔ Run Simulation

▪ 첫번째 read 구간의 시작부분

➔ dout1 ~ dout6 모두 메모리에 저장되어 있는 값이 정상적으로 read

➔ 2-clock delay (Read Latency, 읽기 대기시간)



Using Xilinx IP

Create_BlockMemory .xpr

➤ Create Memory ➔ Block Memory Generator(BMG) ➔ Memory Test (1) ➔ Single Port RAM : spram6 ➔

Simulation (Test Bench) : spram_tb.v 생성 ➔ Run Simulation

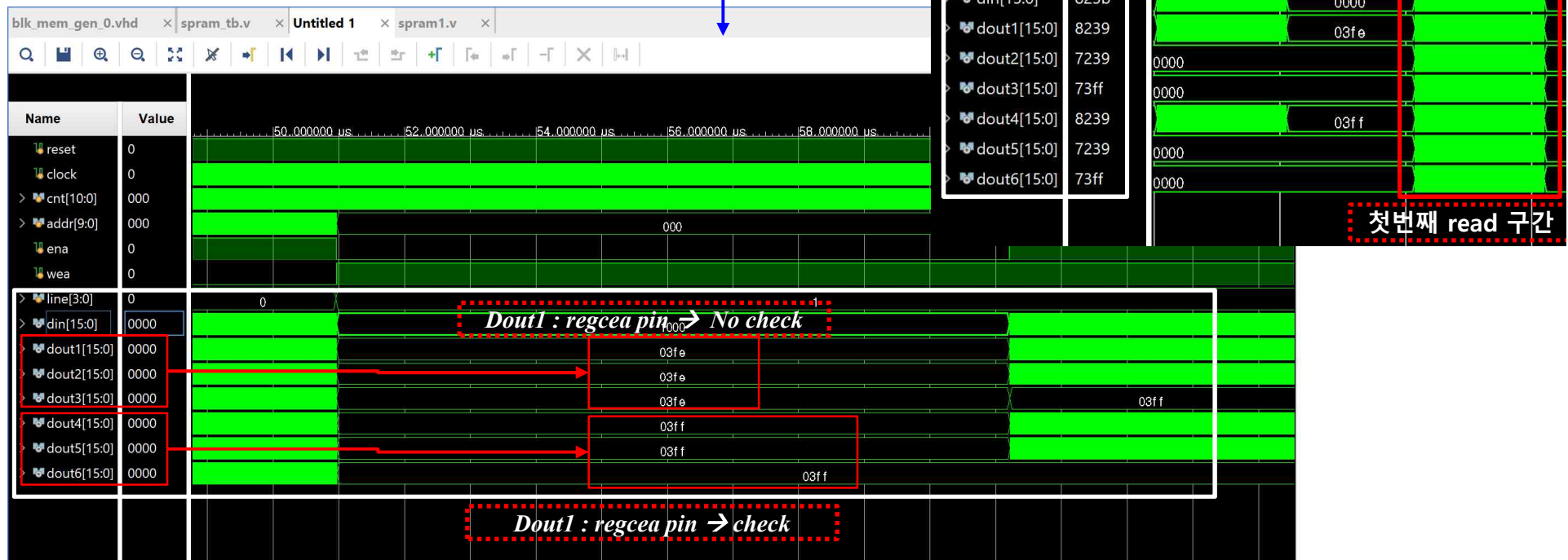
▪ 첫번째 read 구간의 끝부분

➔ dout1 ~ dout6 모두 정상적으로 read

➔ dout1~dout3 : REGCEA Pin "0" 모듈 ➔ 0x03fe까지 data 출력

➔ dout4~dout6 : REGCEA Pin "1" 모듈 ➔ 0x03ff 까지 data 모두 출력

➔ 메모리 생성시 REGCEA Pin ➔ 반드시 추가



Using Xilinx IP

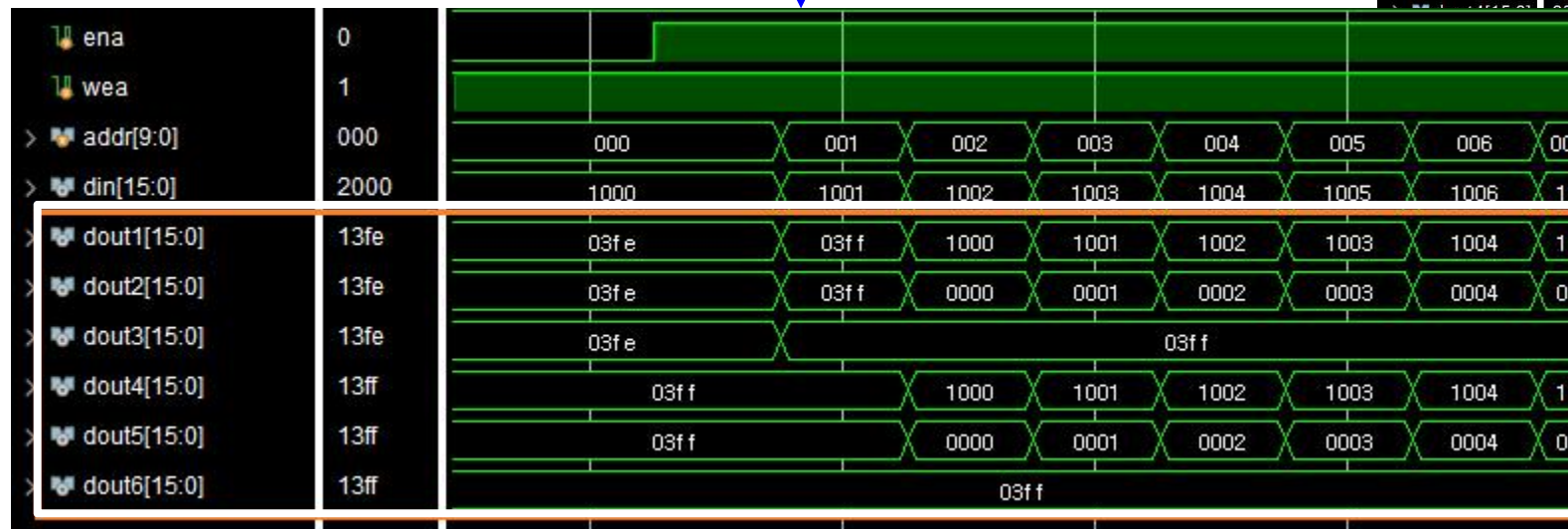
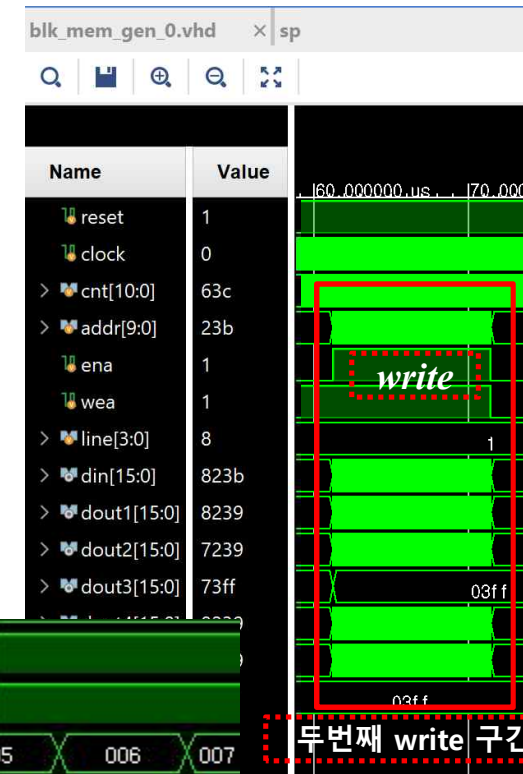
Create_BlockMemory .xpr

➤ Create Memory ➔ Block Memory Generator(BMG) ➔ Memory Test (1) ➔ Single Port RAM : spram6 ➔

Simulation (Test Bench) : spram_tb.v 생성 ➔ Run Simulation

▪ 두번째 write 구간의 첫부분

- ➔ dout1, dout4는 din 데이터가 2-clock delay 후에 나타남
- ➔ dout2, dout5는 메모리에 현재 저장되어 있는 값이 2-clock delay 후에 출력
- ➔ din을 통해 입력(write)되는 데이터는 0x1000, 0x1001, ~ 0x13ff 이고, 현재 메모리에 저장되어 있는 값은 0x0000, 0x0001, ~ 0x03ff
- ➔ dout3, dout6은 이전 값이 그대로
- ➔ 메모리 생성시 REGCEA Pin ➔ 반드시 추가



Using Xilinx IP

Create_BlockMemory .xpr

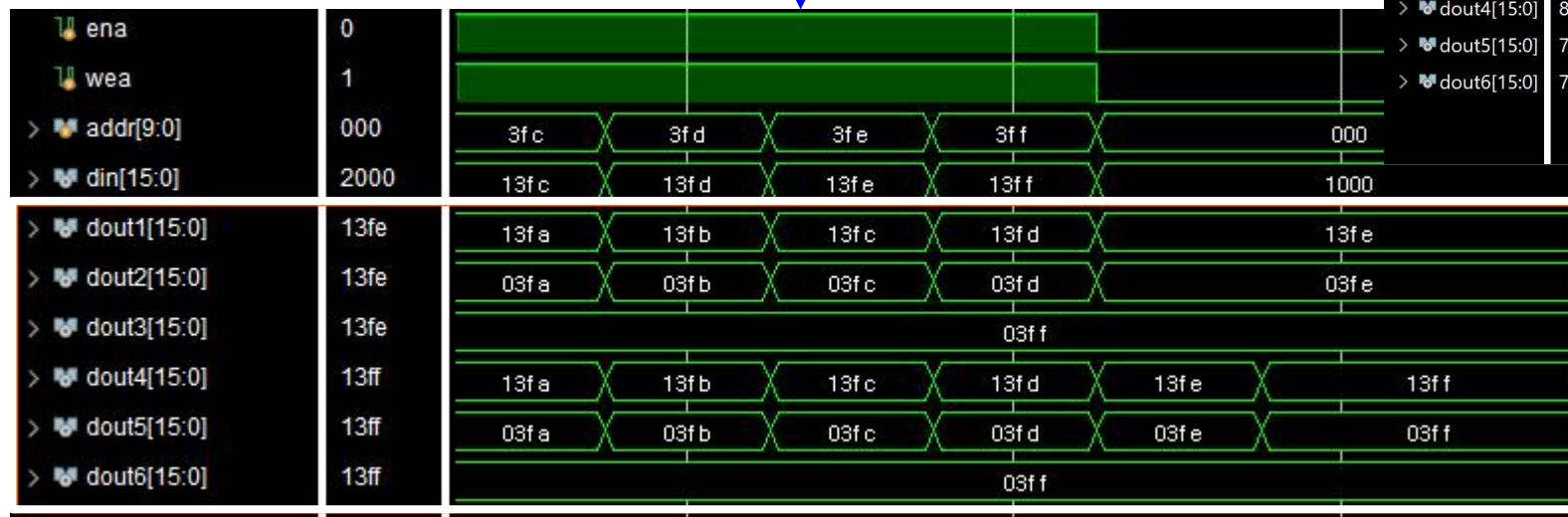
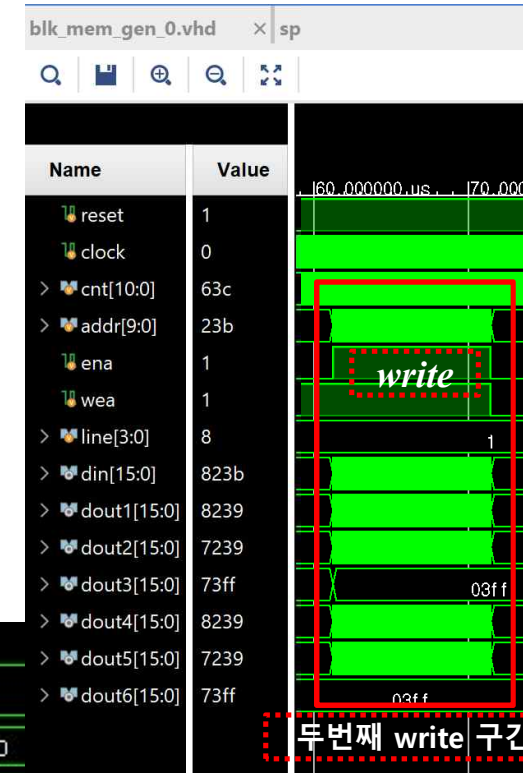
➤ Create Memory ➔ Block Memory Generator(BMG) ➔ Memory Test (1) ➔ Single Port RAM : spram6 ➔

Simulation (Test Bench) : spram_tb.v 생성 ➔ Run Simulation

▪ 두번째 write 구간의 끝부분

➔ dout1, dout2는 마지막 데이터(0x13ff, 0x03ff)가 나오지 않지만, dout4, dout5는 마지막 데이터까지 출력

➔ 메모리 생성시 REGCEA Pin ➔ 반드시 추가



Using Xilinx IP : Create Memory

[Memory Test(2) → Simple Dual Port RAM(SDPRAM)]

➤ Create Memory ➔ Block Memory Generator(BMG) ➔ Memory Test (2) ➔ Simple Dual Port RAM(SDPRAM)

① Memory Generation ➔ Simulation Result (Test Bench)

|| + BRAM_PORTA
|| + BRAM_PORTB

- ✓ Simple Dual Port[(Write Port(A) and Read Port(B)] RAM 은 Two Clock(Write Clock & Read Clock) 사용
- ✓ wea 값에 의해 read/write 설정 ➔ wea가 1이면 write, wea가 0이면 read
- ✓ Operating Mode : Write First, Read First, No Change
 - Write Port와 Read Port가 분리
- ✓ REGCEB pin Option이 있는데 이 핀은 반드시 Check 해서 사용(오류방지)
- ✓ 3개의 SDPRAM 생성[3가지 모드(Write, Read, No change)] 및 시뮬레이션

- Interface Type : Native or AXI4
- Memory Type
 1. Single Port RAM(SPRAM)
: PortA ➔ write / read
 2. Simple Dual Port Ram(SDPRAM)
: PortA ➔ write, PortB ➔ read
 3. True Dual Port RAM(TDPRAM)
: PortA는 write, PortB는 read
 4. Single Port ROM
: PortA ➔ Read
 5. Dual Port ROM
: PortA & PortB ➔ Read

Using Xilinx IP

Create_BlockMemory .xpr

➤ Create Memory → Block Memory Generator(BMG) → Memory Test (2) → Simple Dual Port RAM(SDPRAM)

① Memory Generation → Simulation Result (Test Bench)

② IP Catalog → Memories & Storage Elements – RAMs & ROMs & BRAM – Block Memory Generator 더블

클릭 → 3개의 메모리 생성(1)

1) sdpram1

- ✓ Component Name : **sdpram1**
- ✓ Basic - Memory Type : Simple Dual Port RAM
- ✓ Port A Options – Width : 16, Depth : 1024, Operating Mode : **Write First**
- ✓ Port B Options – Width : 16, Depth : 1024, Operating Mode : **Write First**, **REGCEB pin checked**

Using Xilinx IP

Create_BlockMemory .xpr

➤ Create Memory → Block Memory Generator(BMG) → Memory Test (2) → Simple Dual Port RAM(SDPRAM)

① Memory Generation → Simulation Result (Test Bench)

② IP Catalog → Memories & Storage Elements – RAMs & ROMs & BRAM – Block Memory Generator 더블

클릭 → 3개의 메모리 생성(2)

2) sdpram2

- ✓ Component Name : **sdpram2**
- ✓ Basic - Memory Type : Simple Dual Port RAM
- ✓ Port A Options – Width : 16, Depth : 1024, Operating Mode : **Read First**
- ✓ Port B Options – Width : 16, Depth : 1024, Operating Mode : **Write First**, **REGCEB pin checked**

Component Name: sdpram2

Basic Port A Options Port B Options Other Options Summary

Memory Size

Port A Width: 16 Range: 1 to 4608 (bits)

Port A Depth: 1024 Range: 2 to 1048576

The Width and Depth values are used for Write Operations in Port A

Operating Mode: Read First Enable Port Type: Use ENA Pin

Port A Optional Output Registers

Component Name: sdpram2

Basic Port A Options Port B Options Other Options Summary

Memory Size

Port B Width: 16

Port B Depth: 1024

The Width and Depth values are used for Read Operation in Port B

Operating Mode: Write First Enable Port Type: Use ENB Pin

Port B Optional Output Registers

☒ Primitives Output Register ☐ Core Output Register

☐ SoftECC Output Register ☒ REGCEB Pin

Port B Output Reset Options

☐ RSTB Pin (set/reset pin) Output Reset Value (Hex): 0

☐ Reset Memory Latch Reset Priority: CE (Latch or Register Enable)

READ Address Change B

☐ Read Address Change B

Interface Type: Native

Memory Type: Simple Dual Port RAM

BRAM_PORTA

BRAM_PORTB

regceb

Using Xilinx IP

Create_BlockMemory .xpr

➤ Create Memory → Block Memory Generator(BMG) → Memory Test (2) → Simple Dual Port RAM(SDPRAM)

① Memory Generation → Simulation Result (Test Bench)

② IP Catalog → Memories & Storage Elements – RAMs & ROMs & BRAM – Block Memory Generator 더블

클릭 → 3개의 메모리 생성(3)

PROJECT MANAGER

- Settings
- Add Sources
- Language Templates
- IP Catalog

Block Memory Generator (8.4)

Basic | Port A Options | Port B Options | Other Options | Summary

Memory Size

Port A Width: 16 (Range: 1 to 4608 (bits))

Port A Depth: 1024 (Range: 2 to 1048576)

The Width and Depth values are used for Write Operations in Port A

Operating Mode: No Change (Enable Port Type: Use ENA Pin)

Port A Optional Output Registers

Component Name: sdpram3

Basic | Port A Options | **Port B Options** | Other Options | Summary

Memory Size

Port B Width: 16

Port B Depth: 1024

The Width and Depth values are used for Read Operation in Port B

Operating Mode: Write First (Enable Port Type: Use ENB Pin)

Port B Optional Output Registers

- ☒ Primitives Output Register
- ☐ Core Output Register
- ☐ SoftECC Output Register
- ☒ REGCEB Pin

Port B Output Reset Options

- ☐ RSTB Pin (set/reset pin) Output Reset Value: []
- ☐ Reset Memory Latch Reset Priority: CE (Latch or Register Enable)

READ Address Change B

- ☐ Read Address Change B

3) sdpram3

- ✓ Component Name : **sdpram3**
- ✓ Basic - Memory Type : Simple Dual Port RAM
- ✓ Port A Options – Width : 16, Depth : 1024, Operating Mode : **No Change**
- ✓ Port B Options – Width : 16, Depth : 1024, Operating Mode : **Write First**, **REGCEB pin checked**

BRAM_PORTS

- + BRAM_PORTA
- + BRAM_PORTB
- regceb

➤ Create Memory → Block Memory Generator(BMG) → Memory Test (2) → Simple Dual Port RAM(SDPRAM)

→→→ →→ Simulation (Test Bench) : sdpram_tb.v 생성

① 3개의 메모리 생성(3) → Simulation (Test Bench) : sdpram_tb.v 생성

✓ Memory Test 순서

- ① clka : 100Mhz 동작
- ② clkb : 50Mhz 동작
- ③ Port A : 0 ~ 1023까지 데이터를 Write 합니다.
- ④ Port B : 1024 ~ 2047까지의 데이터를 Read 합니다

Using Xilinx IP

Create_BlockMemory .xpr

➤ Create Memory → Block Memory Generator(BMG) → Memory Test (2) → Simple Dual Port RAM(SDPRAM)

→→→→→ Simulation (Test Bench) : sdpram_tb.v 생성

① 6개의 메모리 생성(6) → Simulation (Test Bench) : spram_tb.v 생성

spram_tb(1/3)

1. ``timescale 1ns / 1ps`

2. `module sdpram_tb();`

3. `reg reset, clka, clkb;`

4. `initial begin`

5. `reset = 0;`

6. `clka = 0;`

7. `clkb = 0;`

8.

9. `#10000 reset = 1;`

10. `end`

11. `always #5 clka = ~clka; // 100 Mhz`

12. `always #10 clkb = ~clkb; // 50 Mhz`

▪ 3 ~ 10 : reset, clocka(100Mhz) , clockb(50Mhz) 생성

```
Project Summary x sdpram1.veo x spram1.veo x
C:/Users/KIM/2023/Create_BlockMemory/Create_BlockMemory.ip_user_files/ip/spram1/spram1.veo

53 // core to be instantiated. Change the instance name and port connections
54 // (in parentheses) to your own signal names.
55
56 //----- Begin Cut here for INSTANTIATION Template -----// INST_TAG
57 spram1 your_instance_name (
58   .clka(clka), // input wire clka
59   .ena(ena), // input wire ena
60   .wea(wea), // input wire [0 : 0] wea
61   .addra(addra), // input wire [9 : 0] addra
62   .dina(dina), // input wire [15 : 0] dina
63   .douta(douta) // output wire [15 : 0] douta
64 );
65 // INST_TAG_END ----- End INSTANTIATION Template -----
66
67 // You must compile the wrapper file spram1.v when simulating
68 // the core, spram1. When compiling the wrapper file, be sure to
69 // reference the Verilog simulation library.
70
```

spram1.veo

```
Project Summary x sdpram1.veo x
C:/Users/KIM/2023/Create_BlockMemory/Create_BlockMemory.ip_user_files/ip/sdpram1/sdpram1.veo

53 // core to be instantiated. Change the instance name and port connections
54 // (in parentheses) to your own signal names.
55
56 //----- Begin Cut here for INSTANTIATION Template -----// INST_TAG
57 sdpram1 your_instance_name (
58   .clka(clka), // input wire clka
59   .ena(ena), // input wire ena
60   .wea(wea), // input wire [0 : 0] wea
61   .addra(addra), // input wire [9 : 0] addra
62   .dina(dina), // input wire [15 : 0] dina
63   .clkb(clkb), // input wire clkb
64   .enb(enb), // input wire enb
65   .regceb(regceb), // input wire regceb
66   .addrb(addrb), // input wire [9 : 0] addrb
67   .doutb(doutb) // output wire [15 : 0] doutb
68 );
69 // INST_TAG_END ----- End INSTANTIATION Template -----
70
```

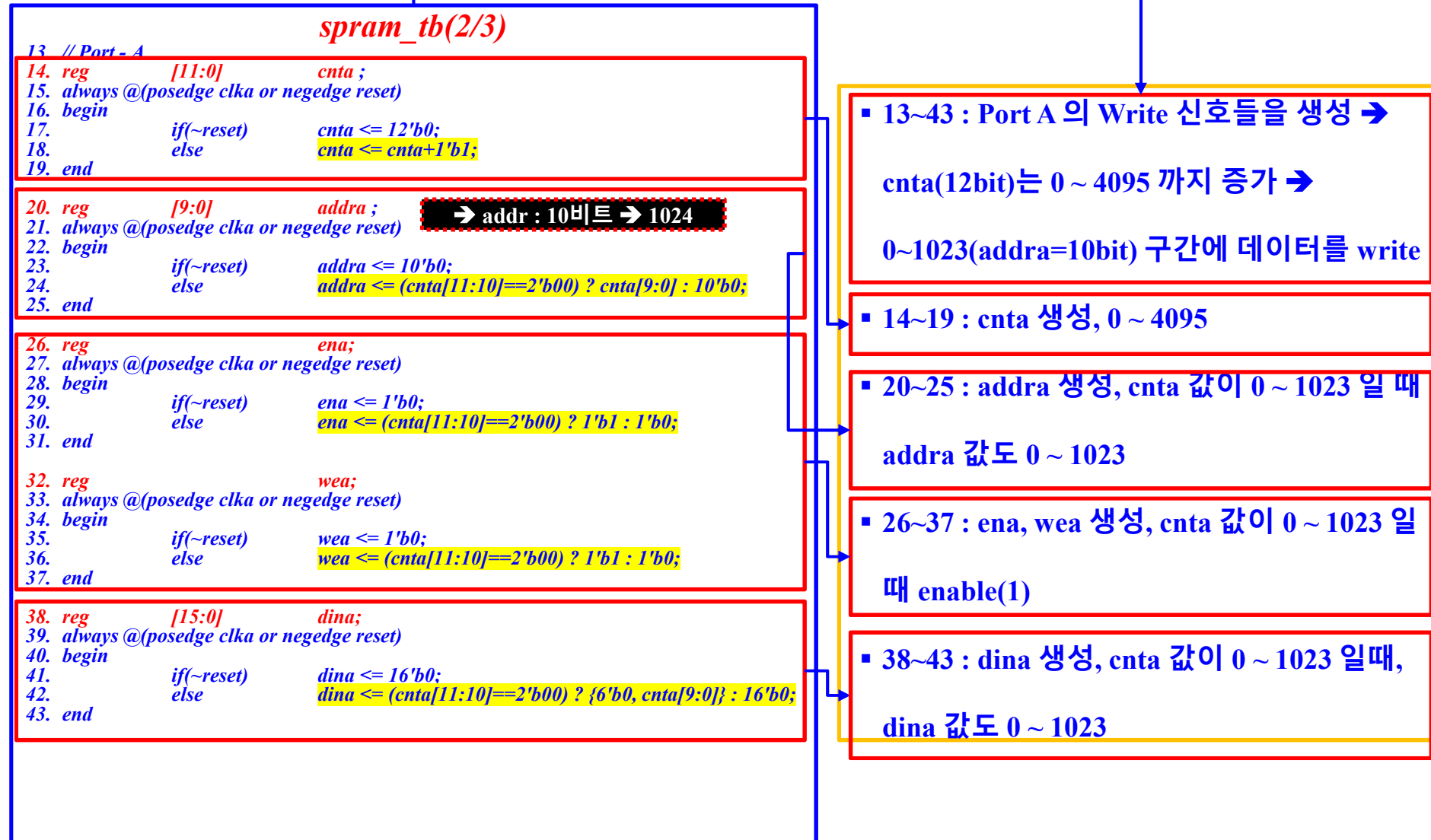
sdpram1.veo

Using Xilinx IP

Create_BlockMemory .xpr

➤ Create Memory ➔ Block Memory Generator(BMG) ➔ Memory Test (2) ➔ Simple Dual Port RAM(SDPRAM)

➔➔➔➔➔ Simulation (Test Bench) : sdpram_tb.v 생성



Using Xilinx IP

Create_BlockMemory .xpr

➤ Create Memory ➔ Block Memory Generator(BMG) ➔ Memory Test (2) ➔ Simple Dual Port RAM(SDPRAM)

spram_tb(3/3)

➔➔➔➔➔ Simulation (Test Bench) : sdpram_tb.v 생성

44. // Port - B

```
45. reg [10:0] cntb ;
46. always @(posedge clkb or negedge reset)
47. begin
48.     if(~reset) cntb <= 11'b0;
49.     else cntb <= cntb+1'b1;
50. end
```

```
51. reg [9:0] addrb ;
52. always @(posedge clkb or negedge reset)
53. begin
54.     if(~reset) addrb <= 10'b0;
55.     else addrb <= cntb[10] ? cntb[9:0] : 10'b0;
56. end
```

```
57. reg enb;
58. always @(posedge clkb or negedge reset)
59. begin
60.     if(~reset) enb <= 1'b0;
61.     else enb <= cntb[10] ? 1'b1 : 1'b0;
62. end
```

```
63. wire [15:0] doutb1 ;
64. sdpram1 sdpram1_u1 ( // write first
65.     .clka(clka), .ena(ena), .wea(wea), .addra(addra),
66.     .dina(dina), .clkb(clkb), .enb(enb), .regceb(1'b1),
67.     .addrb(addrb), .doutb(doutb1)
68. );
```

```
69. wire [15:0] doutb2 ;
70. sdpram2 sdpram2_u1 ( // read first
71.     .clka(clka), .ena(ena), .wea(wea), .addra(addra),
72.     .dina(dina), .clkb(clkb), .enb(enb), .regceb(1'b1),
73.     .addrb(addrb), .doutb(doutb2)
74. );
```

```
75. wire [15:0] doutb3 ;
76. sdpram3 sdpram3_u1 ( // no change
77.     .clka(clka), .ena(ena), .wea(wea), .addra(addra),
78.     .dina(dina), .clkb(clkb), .enb(enb), .regceb(1'b1),
79.     .addrb(addrb), .doutb(doutb3)
80. );
```

81. endmodule

▪ 45~62 : Port B 의 Read 신호들을 생성 ➔ cntb(11bit) 는 0

~ 2047 까지 증가 ➔ 1024 ~ 2047 구간에서 데이터 Read

▪ 45~50 : cntb 생성, 0 ~ 2047

▪ 51~56 : addrb 생성, cntb 값이 1024 ~ 2047 일 때, addrb 값은 0 ~ 1023

▪ 57~62 : enb 생성, cntb 값이 1024 ~ 2047 일 때 enable(1)

▪ 63 : sdpram1 모듈의 read data

▪ 64~68 : sdpram1 모듈

▪ 69 : sdpram2 모듈의 read data

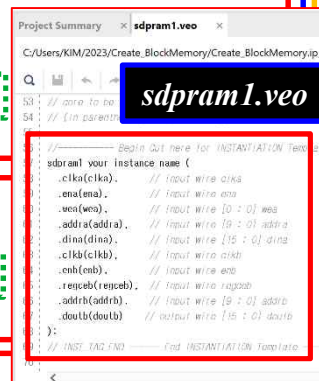
▪ 70~74 : sdpram2 모듈

▪ 75 : sdp_ram3 모듈의 read data

▪ 76~80 : sdpram3 모듈

▪ 66, 72, 78 : .regceb (1'b1), ➔ sdpram3 port에 추가

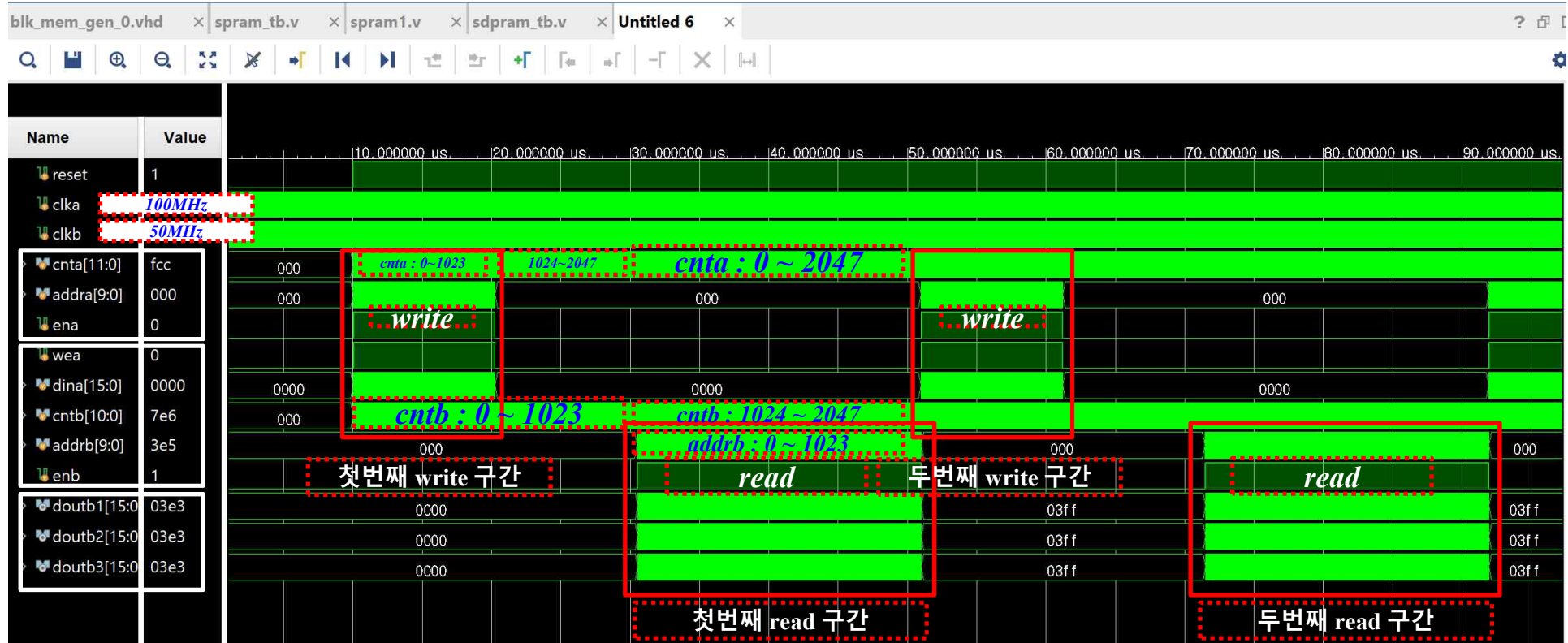
▪ 메모리 생성시 REGCEB pin 을 Check 후 사용



Using Xilinx IP

Create_BlockMemory .xpr

- Create Memory ➔ Block Memory Generator(BMG) ➔ Memory Test (2) ➔ Simple Dual Port RAM(SDPRAM)
- ➔ Simulation (Test Bench) : sdpram_tb.v 생성 ➔ **Run Simulation**



- "run 0.1ms"을 입력해서 0.1ms 동안 진행
- 처음 구간이 Write 구간이고 다음 구간이 Read 구간 ➔ doutb1, doutb2, doutb3 의 값들이 read
- write clock 은 100Mhz, read clock 은 50Mhz ➔ read 구간이 write 구간에 비해서 2 배 긴 것을 확인

Using Xilinx IP

Create_BlockMemory .xpr

➤ Create Memory ➔ Block Memory Generator(BMG) ➔ Memory Test (2) ➔ Simple Dual Port RAM(SDPRAM) ➔

Simulation (Test Bench) : sdpram_tb.v 생성 ➔ **Run Simulation**

▪ 첫번째 write 구간 시작부분 (Port A)

➔ addra : 0x000 에 dina : 0x0000, addra : 0x001 에 dina : 0x0001 차례대로 write

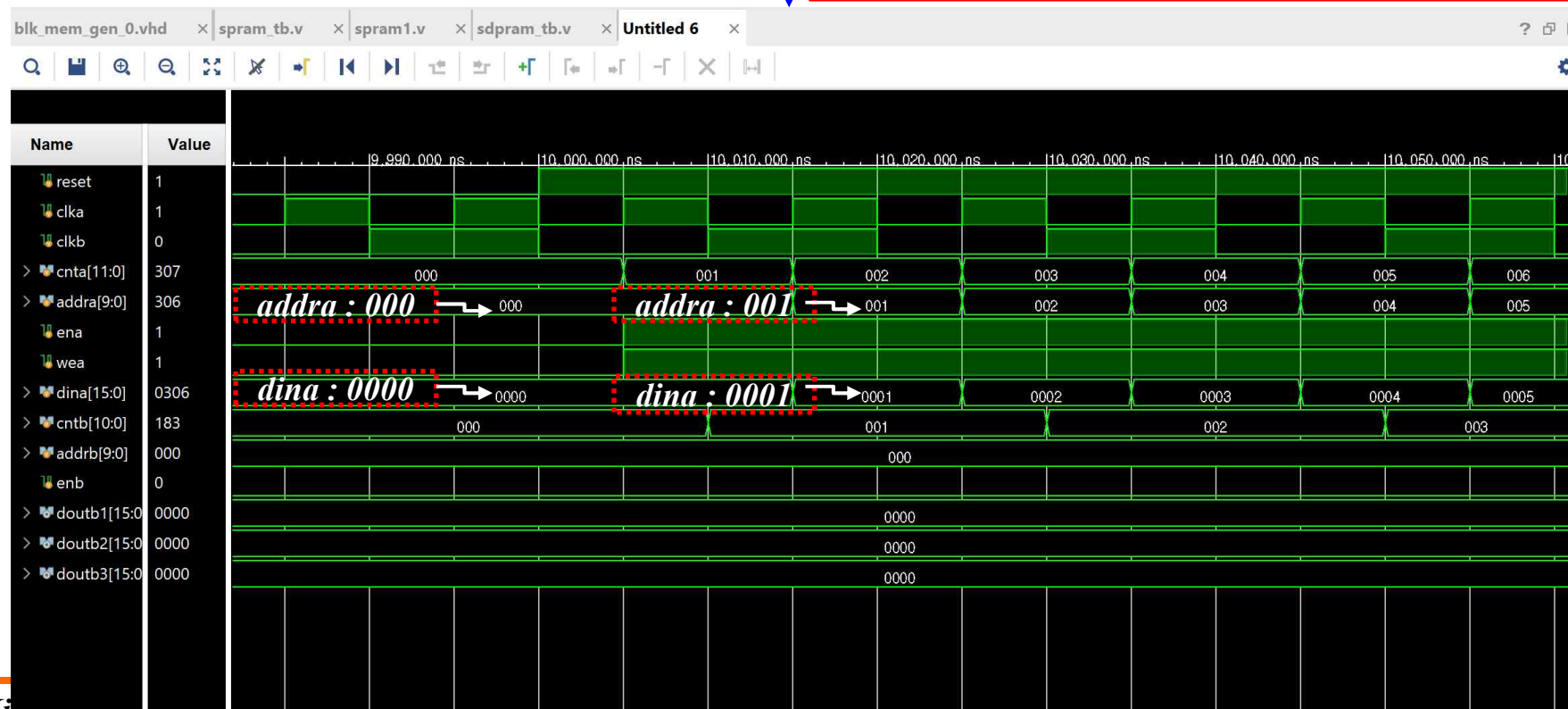
➔ Delay(Latency) 는 Port B (read)만 적용

Component Name: sdpram1

Basic Port A Options Port B Options Other Options **Summary**

Information

Memory Type: Simple Dual Port RAM
Block RAM resource(s) (18K BRAMs): 1
Block RAM resource(s) (36K BRAMs): 0
Total Port B Read Latency (From Rising Edge of Read Clock): 2 Clock Cycle(s)
Address Width A: 10
Address Width B : 10



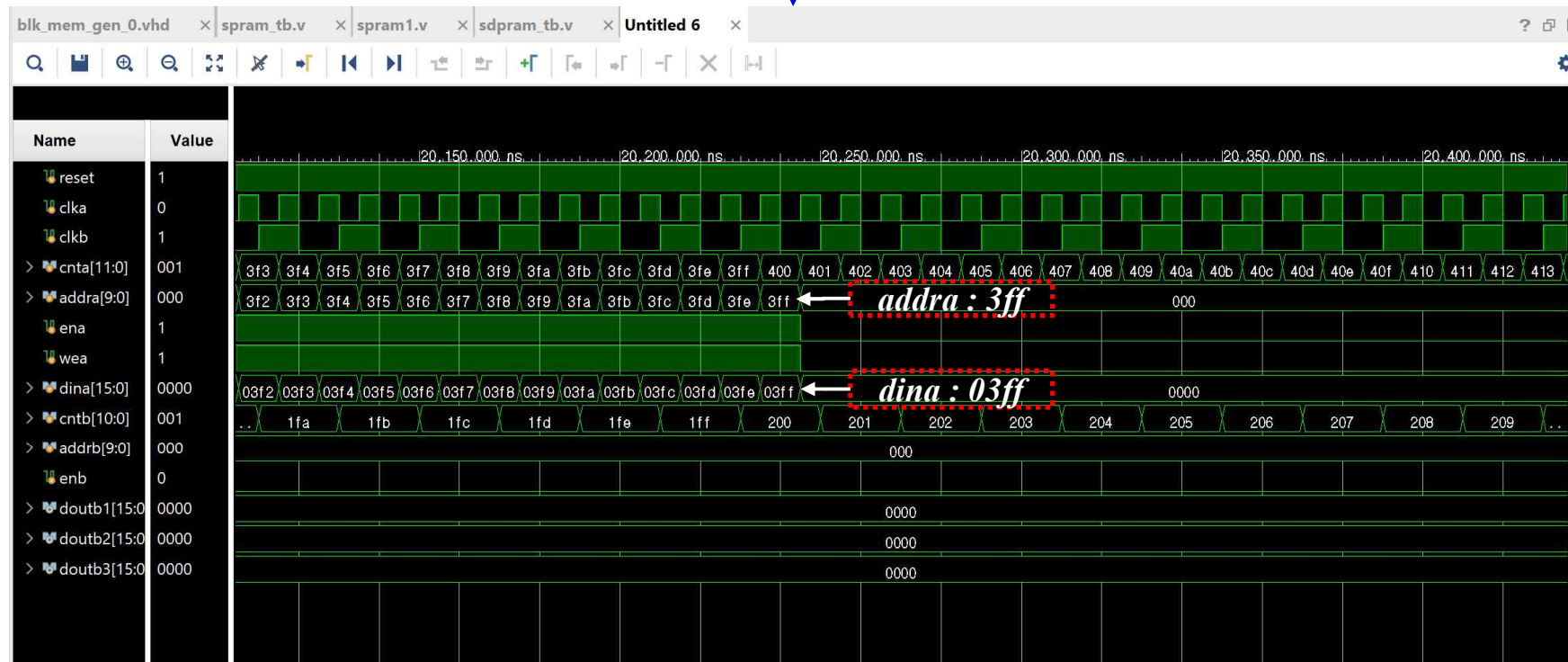
Using Xilinx IP

Create_BlockMemory .xpr

- Create Memory ➔ Block Memory Generator(BMG) ➔ Memory Test (2) ➔ Simple Dual Port RAM(SDPRAM) ➔ Simulation (Test Bench) : sdpram_tb.v 생성 ➔ **Run Simulation**

- 첫번째 write 구간의 끝부분 (Port A)

➔ addra : 0x3ff 에 dina : 0x03ff 까지 write



Using Xilinx IP

Create_BlockMemory .xpr

➤ Create Memory ➔ Block Memory Generator(BMG) ➔ Memory Test (2) ➔ Simple Dual Port RAM(SDPRAM) ➔

Simulation (Test Bench) : sdpram_tb.v 생성 ➔ **Run Simulation**

▪ 첫번째 read 구간의 시작부분

➔ addrb : 0x000 일 때, 2-clock delay 후에 doutb1, doutb2, doutb3 데이터가 모두 0x0000 으로 read 되고, addrb : 0x001 일 때, 2-clock delay 후에 doutb1 ~ doutb3 모두 0x0001 로 read

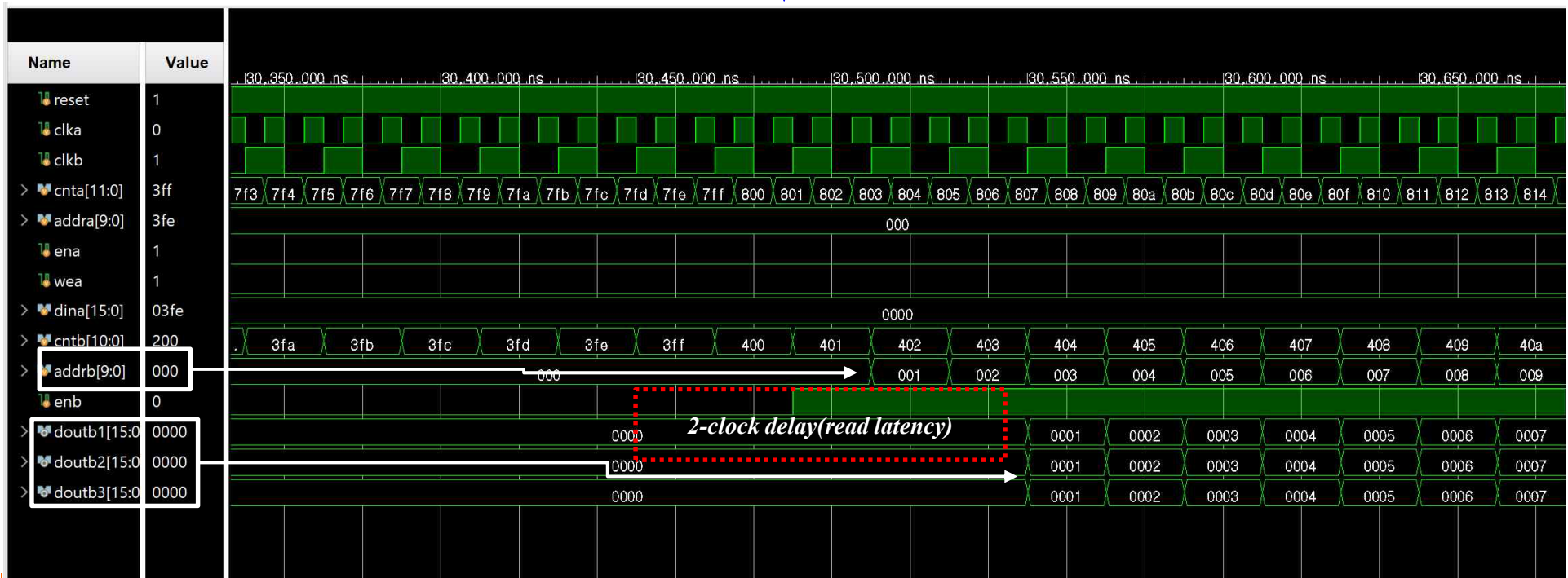
➔ Port-A 는 Operating Mode 를 선택할 수 있지만, PortB 는 항상 Write First Mode 로 동작

Component Name: sdpram1

Basic Port A Options Port B Options Other Options **Summary**

Information

Memory Type: Simple Dual Port RAM
 Block RAM resource(s) (18K BRAMs): 1
 Block RAM resource(s) (36K BRAMs): 0
Total Port B Read Latency (From Rising Edge of Read Clock): 2 Clock Cycle(s)
 Address Width A: 10
 Address Width B: 10



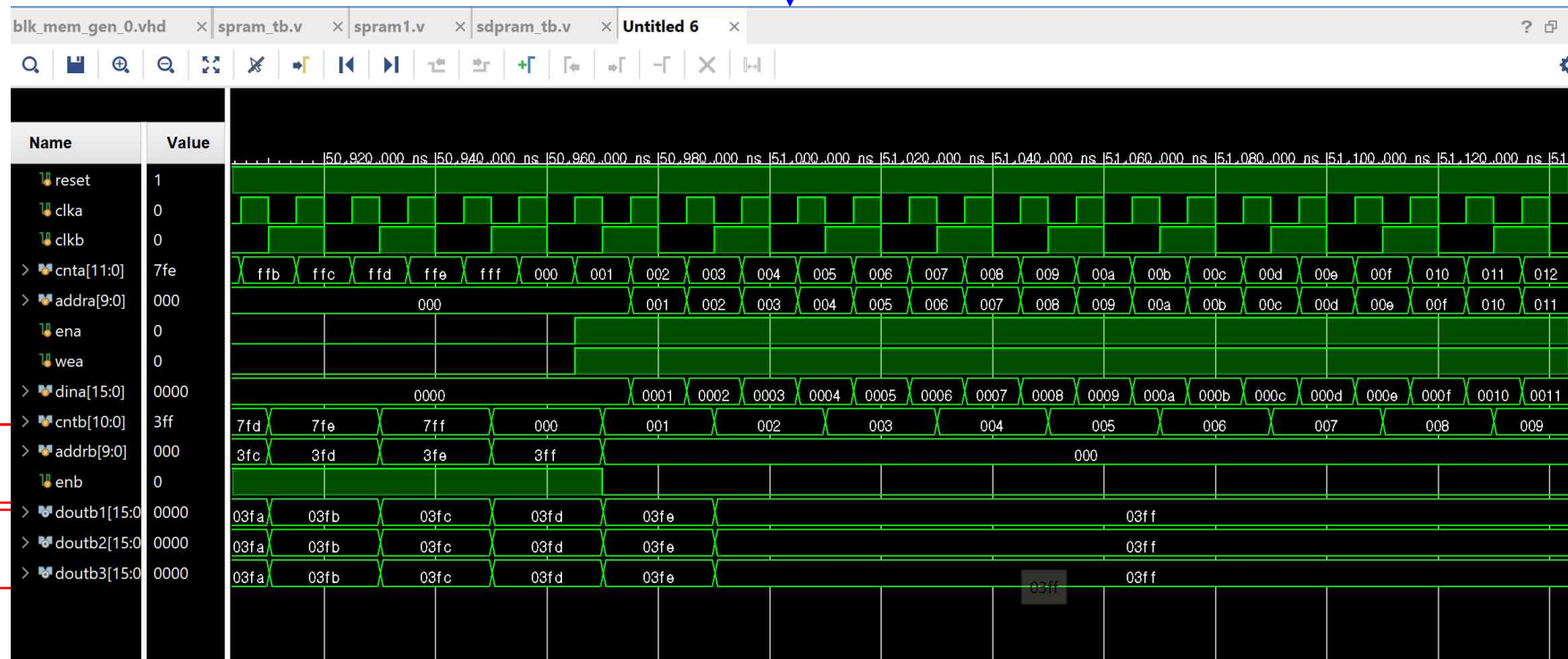
Using Xilinx IP

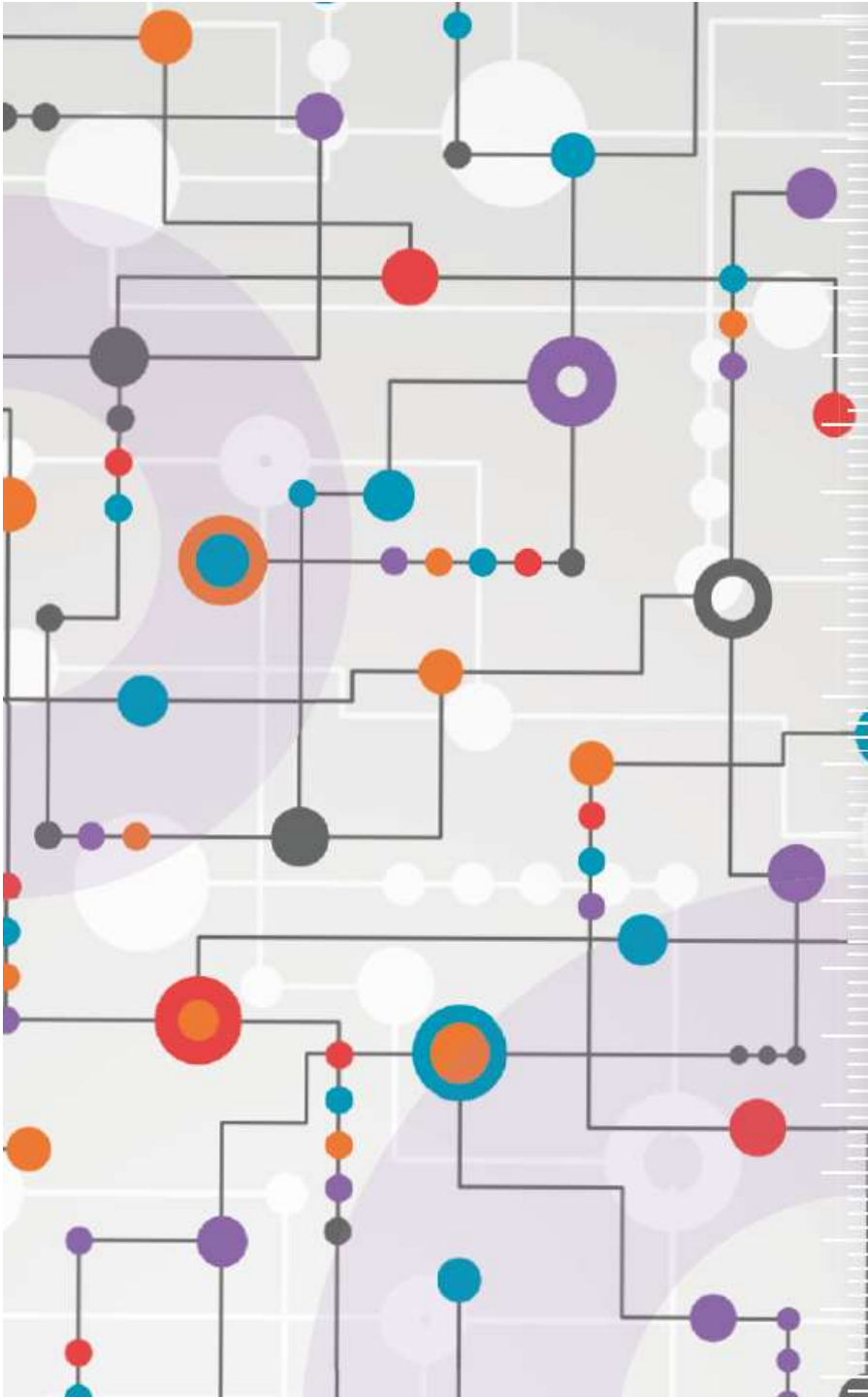
Create_BlockMemory .xpr

- Create Memory ➔ Block Memory Generator(BMG) ➔ Memory Test (2) ➔ Simple Dual Port RAM(SDPRAM) ➔ Simulation (Test Bench) : sdpram_tb.v 생성 ➔ **Run Simulation**

▪ 첫번째 read 구간의 끝부분

- ➔ 0x03ff 까지 데이터가 read : REGCEB 핀에 “1”
➔ REGCEB(A) 핀이 ‘0’ 인 경우 0x03fe 까지 read





수고하셨습니다.